COURSE REFERENCE / PRINT EDITION

RxJS Deep Dive

Stream Signature Reference

A* => B*

TYPE SHAPE + BEHAVIOR + EXECUTION DETAILS

Cω-inspired stream shorthand that translates into ordinary TypeScript types. It is explanatory notation, not syntax accepted by the TypeScript compiler.

147 entries across 10 operator families

112 course-list entries + 35 supplements

120 current APIs + 27 deprecated APIs with replacements

RxJS 7.8.2 / Edition 1.0 / 2026-10-12

Course direction: Hans Schenker

https://github.com/hansschenker/rxjs-ts-shorthand-notation

1. These definitions and teaching notes were created with the help of GPT-6 Astra, under the course direction of Hans Schenker.

Contents and reading guide

147 entries. The category links and alphabetical index open the corresponding pages. A SUPPLEMENT label identifies an entry added to the supplied course list.

01 Creation Operators 4

02 Join Creation Operators 13

03 Transformation Operators 17

04 Filtering Operators 32

05 Join Operators 41

06 Multicasting Operators 51

07 Error Handling Operators 56

08 Utility Operators 58

09 Conditional and Boolean Operators 64

10 Mathematical and Aggregate Operators 66

Read the shape and the policy together

- The cards show selected common forms. Callback indices, some optional parameters, and the exhaustive overload sets live in the linked API and pinned source.
- Creation functions return Observables directly. Most pipeable functions return transformations used by pipe; partition and connectable forms have explicitly different result shapes.
- Unless a card says otherwise, next, error, complete, and unsubscription are separate concerns. Unsubscription is not a complete notification.
- Examples omit imports and generally show Observable expressions, not subscribe calls. Their next/complete comments describe a subscription allowed to finish; external-request outcomes remain conditional.
- Durations normally use milliseconds with asyncScheduler. A supplied scheduler can provide a different or virtual clock. Integer counts and positive concurrency limits are the teaching defaults.
- null and undefined are ordinary emitted values. of(undefined) emits once; EMPTY emits no next values.
- An operator factory builds a description. Work starts through subscription, although argument expressions and eager input creation may already have run.
- Shorthand describes type shape. Read the adjacent behavior policy and execution details before claiming two operators are equivalent.

Read the notation

Cω-inspired stream shorthand that translates into ordinary TypeScript types. It is explanatory notation, not syntax accepted by the TypeScript compiler.

Notation	Read it as
`A, B`	Input and output payload types. S is accumulated state; K is a key; O is an opening-signal payload; R is a registration token; D is a default-value type.
`A*`	Observable<A>: a stream with zero or more next values per subscription. The type does not promise values, completion, a clock, or shared execution.
`A? / A! / A+`	Optional occurrence refinements: zero or one / exactly one / one or more next values. State the lifetime and termination assumptions. Ordinary Observable<A> does not enforce these bounds.
`A[] / (A[])*`	An array value / an Observable whose next values are arrays.
`[A*, B*] / [A, B]*`	A tuple of two sources / a stream of two-value tuples.
`(A*)*`	An outer stream whose next values are inner streams.
`A => B`	A value function; in TypeScript write (value: A) => B.
`A* => B*`	A source-to-result transformation; in TypeScript write (source: Observable<A>) => Observable<B>.
`Input<A>`	ObservableInput<A>: Observable, promise-like, iterable, async iterable, array-like, interoperable Observable, or readable-stream-like input.
`(A | B)*`	A stream whose next values can belong to either payload type.
`A & B`	A value satisfying both structural requirements; used for a group stream with a key property.
`never*`	No next payload values. Completion, error, or remaining open must be described separately.
`Notice<A>`	ObservableNotification<A>: { kind: "N"; value: A } | { kind: "E"; error: unknown } | { kind: "C" }.
`param?: Type`	An optional configuration parameter. This question mark is parameter syntax, not the stream-cardinality suffix.

01 / 24 entries

Creation Operators

Start from values, callbacks, events, requests, or a clock. Read when work begins and how the subscription ends.

ajax

CREATION / CURRENT / COURSE LIST

```
ajax(request: string | AjaxConfig): AjaxResponse<A>*
```

Start an XMLHttpRequest on subscription and emit response objects containing a payload of type A. With the default progress settings, a successful request emits its response and completes.

- Each subscription creates a request. Defaults include GET, asynchronous operation, and JSON response decoding.
- Network errors, timeouts, and HTTP statuses of 400 or above use the error channel.
- Unsubscribing aborts an unfinished request.
- Enabling upload or download progress adds response-shaped progress emissions; ajax is therefore not universally a one-value stream.

```
ajax<{ name: string }>('/api/profile')
// On a successful default request: next an AjaxResponse whose response is the profile; complete.
```

Scope. A names the decoded response payload, not the entire response wrapper; the type argument performs no runtime validation. ajax.getJSON<A>(url): A* selects only the response body. GET/POST/PUT/PATCH/DELETE helpers and AjaxConfig provide request options.

Related: fromFetch / defer

`rxjs/ajax` | API docs | Pinned source | ID: ajax

bindCallback

CREATION / CURRENT / COURSE LIST

```
bindCallback(fn: (arg: A, callback: B => void) => void): A => B*
```

Turn a callback API into a function that returns a stream of its result. Calling the bound function prepares that stream; its first subscription invokes the original function.

- For a normal single-result callback, emit its result and complete when the callback resolves.
- Subscribers to the same returned Observable share one invocation and receive its cached result, including late subscribers. Calling the bound function again creates a separate result stream.
- Unsubscribing removes this observer; it does not supply a cancellation mechanism to the callback API.

```
const double = (n: number, done: (result: number) => void) => done(n * 2);
bindCallback(double)(3) // next: 6; complete synchronously
```

Scope. Shown for one input argument and one callback result. Real signatures preserve arbitrary argument tuples; several result arguments become one tuple, while no result arguments produce one undefined value. An optional scheduler controls invocation and delivery; a result-selector form also exists.

Related: bindNodeCallback / fromEventPattern

`rxjs` | API docs | Pinned source | ID: bindCallback

bindNodeCallback

CREATION / CURRENT / COURSE LIST

```
bindNodeCallback(fn: (arg: A, callback: (error: unknown, value: B) => void) => void): A => B*
```

Adapt a Node-style error-first callback. A successful callback supplies the result value; an error argument terminates the stream through the error channel.

- Only null or undefined in the error slot means success; other values count as errors.
- The original function runs on the first subscription to each returned Observable. That Observable shares and caches its callback result for later subscribers.
- This adapter does not automatically cancel the underlying operation when a subscriber unsubscribes.

```
const lengthOf = (s: string, done: (err: Error | null, n: number) => void) => done(null, s.length);
bindNodeCallback(lengthOf)('RxJS') // next: 4; complete synchronously
```

Scope. Shown for one input and one success result. Real signatures preserve argument tuples; multiple success results become one tuple, and no success result produces undefined. Optional scheduler and result-selector forms are available.

Related: bindCallback / from

rxjs | API docs | Pinned source | ID: bindNodeCallback

defer

CREATION / CURRENT / COURSE LIST

```
defer(factory: () => Input<A>): A*
```

Run the factory separately for each subscription, convert its result to an Observable, and forward that execution.

- The factory runs when subscribed, not when defer is called.
- A thrown factory error becomes an error notification.
- A promise created inside the factory starts at subscription time. Returning an already-created promise does not restart its work.

```
const value$ = defer(() => of(Math.random()));
// Each subscription draws a new number, emits it, and completes.
```

Scope. The factory can return any ObservableInput, including an Observable, promise, array, or iterable.

Related: from / iif / using

rxjs | API docs | Pinned source | ID: defer

empty

CREATION / DEPRECATED / COURSE LIST

```
empty(scheduler?: SchedulerLike): never*
```

Create a stream that completes without emitting a value.

- Without a scheduler, this returns EMPTY and completes synchronously on subscription.
- With a scheduler, schedule the completion notification; unsubscribing first cancels that scheduled completion.

Use now: EMPTY; for scheduled completion, scheduled([], scheduler).

```
empty() // no next values; complete synchronously
```

Scope. This function is deprecated in RxJS 7.8.2. EMPTY expresses the usual immediate form.

Related: EMPTY / NEVER / throwError

rxjs | API docs | Pinned source | ID: empty

from

CREATION / CURRENT / COURSE LIST

```
from(input: Input<A>): A*
```

Convert an Observable-compatible input into a stream of its values. The input kind determines how those values become available.

- Arrays and synchronous iterables emit their items synchronously by default, then complete when exhausted.
- A promise emits its resolved value and completes, or reports its rejection as an error. Its underlying work may already have started.
- An existing Observable retains its execution behavior. Unsubscribing from a promise conversion stops delivery but does not cancel the promise itself.

```
from([1, 2, 3]) // next: 1, 2, 3; complete synchronously
```

Scope. Input<A> includes Observable, interoperable Observable, promise-like, array-like, iterable, async iterable, and readable-stream-like inputs. The scheduler-argument overload is deprecated; use scheduled(input, scheduler).

Related: of / scheduled / defer

`rxjs` | API docs | Pinned source | ID: from

fromEvent

CREATION / CURRENT / COURSE LIST

```
fromEvent(target: EventTarget, eventName: string, options?: AddEventListenerOptions): A*
```

Turn events from a target into values. In the common DOM form, each event object becomes one emitted value.

- Each subscription installs its own listener; unsubscription removes that listener.
- An event source does not complete merely because no more events arrive.
- DOM options such as once affect the listener; once does not send an RxJS complete notification.

```
fromEvent<MouseEvent>(document, 'click')
// next: each click event; unsubscribe removes this listener
```

Scope. Shown for a DOM EventTarget with an explicitly selected event type A. Node-style and jQuery-style emitters and collections of targets are also supported. Multiple event-handler arguments are emitted as one array; result-selector forms can project them.

Related: fromEventPattern / takeUntil

`rxjs` | API docs | Pinned source | ID: fromEvent

fromEventPattern

CREATION / CURRENT / COURSE LIST

```
fromEventPattern(addHandler: (handler: A => void) => R, removeHandler?: (handler: A => void,
  token: R) => void): A*
```

Adapt an event API by supplying the functions that attach and detach a handler. Every handler call supplies an emission.

- addHandler runs for each subscription. Its return value is a registration token, passed to removeHandler together with the same handler during teardown.
- If removeHandler is omitted, this adapter cannot detach the registration for you.
- Events continue until the subscription ends; the adapter has no automatic completion signal.

```
fromEventPattern<Event>(
  handler => document.addEventListener('visibilitychange', handler),
  handler => document.removeEventListener('visibilitychange', handler)
) // next: visibility events; unsubscribe detaches the handler
```

Scope. Shown for a handler receiving one A and a registration token R, which may be void. Several handler arguments become one array, while zero handler arguments become an empty array. An optional result selector can map the handler arguments.

Related: fromEvent / bindCallback

rxjs | API docs | Pinned source | ID: fromEventPattern

generate

CREATION / CURRENT / COURSE LIST

```
generate(options: {
  initialState: S,
  condition?: S => boolean,
  iterate: S => S,
  resultSelector: S => A,
  scheduler?: SchedulerLike
}): A*
```

Generate a sequence by repeatedly testing a state, projecting a value, and advancing the state. Stop when the condition becomes false.

- The condition is checked before emitting from the initial state and before every later state.
- Without a scheduler, the loop runs synchronously; an unbounded loop can occupy the current call stack until unsubscribed or interrupted by an error.
- A supplied scheduler schedules iteration. Each subscription starts again from initialState.

```
generate({ initialState: 1, condition: n => n <= 3, iterate: n => n + 1, resultSelector: n => n * n
})
// next: 1, 4, 9; complete synchronously
```

Scope. Without resultSelector, emit the state itself: S*. Omitting condition generates without a condition-based stopping point. The older positional-argument overloads are deprecated; the options-object form is current.

Related: range / expand / scan

rxjs | API docs | Pinned source | ID: generate

interval

CREATION / CURRENT / COURSE LIST

```
interval(period?: number, scheduler?: SchedulerLike): number*
```

Emit an increasing counter at a recurring interval, starting with 0 after the first period.

- Defaults are period = 0 and asyncScheduler; a negative period is treated as 0.
- Each subscription starts its own counter and scheduled work.
- It has no natural completion. Unsubscribing cancels its scheduled work.

```
interval(1000).pipe(take(3))
// next: 0, 1, 2 on successive one-second periods; then complete
```

Scope. Periods use milliseconds with the default scheduler; actual delivery follows the selected scheduler and host timing.

Related: timer / animationFrames

rxjs | API docs | Pinned source | ID: interval

of

CREATION / CURRENT / COURSE LIST

```
of(...values: A[]): A*
```

Emit the supplied argument values in order, then complete. Each argument stays a single value.

- By default, emission and completion happen synchronously during subscription.
- Arrays, promises, null, and undefined are emitted as values; of does not unpack them.
- With no arguments, complete without a next value.

```
of([1, 2], [3]) // next: [1, 2], [3]; complete synchronously
```

Scope. For heterogeneous arguments, A is their union. Scheduler-argument overloads are deprecated; use scheduled(values, scheduler) when scheduling these values is intended.

Related: from / EMPTY

rxjs | API docs | Pinned source | ID: of

range

CREATION / CURRENT / COURSE LIST

```
range(start: number, count: number): number*
```

Emit count numbers, starting at start and increasing by 1, then complete.

- The common form emits synchronously during subscription.
- A count of zero or less produces an empty completion; use nonnegative integer counts for the intended counting contract.
- Downstream unsubscription stops the loop before later values are emitted.

```
range(3, 3) // next: 3, 4, 5; complete synchronously
```

Scope. The single argument is the count: range(3) starts at 0 and emits 0, 1, 2. The scheduler-argument overload is deprecated; scheduled(range(start, count), scheduler) schedules observable subscription and delivery.

Related: generate / of / scheduled

rxjs | API docs | Pinned source | ID: range

throwError

CREATION / CURRENT / COURSE LIST

```
throwError(errorFactory: () => unknown): never*
```

Create a stream that reports an error as soon as it is subscribed.

- The factory runs for each subscription, so an error object can be created at the point of execution.
- The result emits no next values and terminates with error, rather than complete.

```
throwError(() => new Error('Failed')) // no next values; error: Failed
```

Scope. The overload accepting an error value directly and the scheduler-argument overload are deprecated. The error factory form is current.

Related: EMPTY / NEVER / catchError

rxjs | API docs | Pinned source | ID: throwError

timer

CREATION / CURRENT / COURSE LIST

```
timer(due: number | Date, scheduler?: SchedulerLike): number*
```

Wait until the due time, emit 0, and complete. A separate repeating form continues with an increasing counter.

- A numeric due time is relative to subscription; a Date is interpreted against scheduler.now(). Past due times are treated as zero delay.
- The default scheduler is asyncScheduler, so timer(0) schedules its emission rather than behaving like synchronous of(0).
- timer(startDue, period, scheduler?) emits 0 at the start time and then 1, 2, ... at that period; a nonnegative period makes it recurring.
- Unsubscribing cancels the scheduled work. The repeating form has no natural completion.

```
timer(1000) // after the scheduled delay: next 0; complete
```

Scope. Shown for the one-shot overload, whose runtime value is 0; the public TypeScript payload type is number. The repeating overload also returns number*. A negative interval duration is treated as a one-shot timer in 7.8.2.

Related: interval / delay / takeUntil

rxjs | API docs | Pinned source | ID: timer

iif

CREATION / CURRENT / COURSE LIST

```
iif(condition: () => boolean, whenTrue: Input<A>, whenFalse: Input<B>): (A | B)*
```

Choose one of two inputs at subscription time and forward that chosen execution.

- Evaluate condition for every subscription and subscribe only to the selected input.
- Constructing the branch arguments still follows normal JavaScript evaluation. Wrap branch work in defer when its creation must also wait.
- A thrown condition error goes to the error channel.

```
iif(() => true, of('yes'), of('no')) // next: 'yes'; complete
```

Scope. Both branches accept ObservableInput. The result payload is the union of their value types.

Related: defer / EMPTY

rxjs | API docs | Pinned source | ID: iif

animationFrames

CREATION / CURRENT / SUPPLEMENT

```
animationFrames(timestampProvider?: TimestampProvider): ({ timestamp: number, elapsed: number
  })*
```

Emit a timing record on each animation frame. elapsed measures the time since this subscription began.

- Uses requestAnimationFrame; each subscription has its own frame loop and starting time.
- Without a custom provider, timestamp comes from the animation-frame callback and elapsed uses performance timing.
- A custom TimestampProvider supplies the current time used for both fields. Unsubscription cancels the pending frame request.

```
animationFrames().pipe(take(2))
// next: timing records for two animation frames; then complete
```

Scope. Requires an environment with animation-frame support. It does not complete by itself.

Related: interval / timestamp

rxjs | API docs | Pinned source | ID: animationFrames

EMPTY

CONSTANT / CURRENT / SUPPLEMENT

```
EMPTY: never*
```

An Observable constant that completes immediately without emitting a value.

- Each subscription receives a synchronous complete notification.
- It differs from of(undefined), which emits one undefined value before completing.

```
EMPTY // when subscribed: no next values; complete synchronously
```

Scope. Use this constant for immediate empty completion; scheduled([], scheduler) makes completion scheduled.

Related: empty / NEVER / throwError

rxjs | API docs | Pinned source | ID: EMPTY

fromFetch

CREATION / CURRENT / SUPPLEMENT

```
fromFetch(input: string | Request, init?: RequestInit): Response*
```

Start a fetch request per subscription. In the default form, emit the Response when it is available and then complete; reading its body is a separate step.

- HTTP error status codes still produce Response values; inspect response.ok or response.status when deciding whether to treat them as errors.
- Network failures and abort failures use the error channel.
- Unsubscribing while the fetch is pending aborts it. In the default form, automatic abort stops once the Response has been delivered.
- The selector form keeps the request abortable while the selected input, such as response.json(), is being consumed.

```
fromFetch('/api/profile', { selector: response => response.json() })
// On successful JSON decoding: next the decoded body; complete
```

Scope. With init.selector: Response => Input<A>, the result is A*. Use that form to include body consumption in the subscription's lifetime; the return type of response.json() itself does not validate a domain schema.

Related: ajax / switchMap / catchError

rxjs/fetch | API docs | Pinned source | ID: fromFetch

NEVER

CONSTANT / CURRENT / SUPPLEMENT

```
NEVER: never*
```

An Observable constant that sends no next, error, or complete notifications.

- A subscription remains open until explicitly unsubscribed or ended by another operator.
- The type never* says only that there are no next values; the absence of completion is this constant's behavior.

```
NEVER.pipe(takeUntil(timer(1000)))
// no next values; complete when the timer emits
```

Related: EMPTY / never / takeUntil

rxjs | API docs | Pinned source | ID: NEVER

never

CREATION / DEPRECATED / SUPPLEMENT

```
never(): never*
```

Return NEVER: a stream that sends no notifications and has no natural termination.

- The returned subscription stays open until it is unsubscribed or another operator ends it.
- This legacy function and the NEVER constant describe the same behavior.

Use now: NEVER

```
never() // when subscribed: no next, no error, no complete
```

Scope. The function is deprecated in RxJS 7.8.2.

Related: NEVER / EMPTY

rxjs | API docs | Pinned source | ID: never

pairs

CREATION / DEPRECATED / SUPPLEMENT

```
pairs(object: Record<string, A>, scheduler?: SchedulerLike): [string, A]*
```

Emit the own enumerable string-keyed entries of an object as key-value tuples, then complete.

- The implementation calls Object.entries when pairs is called, so the entry array is captured before subscription.
- Without a scheduler, emit the entries synchronously on subscription. Symbol keys and inherited properties are not included.
- The tuples hold the original property values; referenced objects are not deeply copied.

Use now: from(Object.entries(object)); use scheduled(Object.entries(object), scheduler) for scheduled delivery.

```
pairs({ a: 1, b: 2 }) // next: ['a', 1], ['b', 2]; complete
```

Scope. Shown for a homogeneous string-keyed record. The legacy API has additional input overloads and an optional scheduler. Its implementation uses Object.entries rather than consuming an iterable protocol.

Related: from / scheduled

rxjs | API docs | Pinned source | ID: pairs

scheduled

CREATION / CURRENT / SUPPLEMENT

```
scheduled(input: Input<A>, scheduler: SchedulerLike): A*
```

Convert an input into an Observable while using the supplied scheduler to control its consumption and delivery.

- For Observable inputs, subscription and notifications are scheduled. For arrays and synchronous iterables, element production is scheduled.
- The scheduler determines timing; queueScheduler can run work synchronously, while other schedulers defer it.
- Scheduling a promise conversion does not defer the work that created the already-existing promise, and unsubscription does not cancel that promise.

```
scheduled([1, 2], asyncScheduler)
// scheduled next: 1, 2; then scheduled complete
```

Scope. Accepts ObservableInput, including async iterables and readable streams. This replaces many deprecated scheduler-argument overloads on creation functions.

Related: from / observeOn / subscribeOn

rxjs | API docs | Pinned source | ID: scheduled

using

CREATION / CURRENT / SUPPLEMENT

```
using(resourceFactory: () => (Unsubscribable | void), observableFactory: (Unsubscribable |
  void) => (Input<A> | void)): A*
```

Create a resource for a subscription and pass it to a factory that supplies that subscription's values. Dispose the resource when the established execution is torn down.

- Both factories run separately for each subscription. The resource exposes an unsubscribe method; the factory may also return no resource.
- After setup succeeds, completion, error, or downstream unsubscription disposes the resource. A void Observable-factory result acts like EMPTY.
- In 7.8.2, cleanup is registered after observableFactory returns: if that factory throws after allocating the resource, provide cleanup for that failure path yourself.

```
using(
  () => ({ unsubscribe: () => console.log('disposed') }),
  () => of(1, 2)
) // next: 1, 2; complete; resource cleanup logs 'disposed'
```

Scope. The actual Observable factory can return any ObservableInput, or void. The displayed resource contract is the API's Unsubscribable | void type.

Related: defer / finalize

rxjs | API docs | Pinned source | ID: using

webSocket

CREATION / CURRENT / SUPPLEMENT

```
webSocket(urlOrConfig: string | WebSocketSubjectConfig<A>): WebSocketSubject<A>
```

Create a duplex WebSocket endpoint: subscribing receives A values, while calling next sends A values to the server.

- The first subscription opens the socket; subscribers to the same WebSocketSubject share that connection. When its last observer unsubscribes, it closes the socket.
- Outgoing values sent before the socket opens are queued. Sending a value does not also emit it locally as an incoming value.
- The default serializer uses JSON.stringify and the default deserializer uses JSON.parse; configure them for a different wire format.
- A clean socket close completes its observers; an unclean close or socket error errors them.

```
const messages = webSocket<{ text: string }>('wss://example.test/messages');
// messages.subscribe(...): receive server messages
// messages.next({ text: 'hello' }): send a message
```

Scope. WebSocketSubject<A> is both an A* read side and an Observer<A> write side, so reducing the return type to A* would hide its duplex behavior. Configuration supports custom serialization, deserialization, lifecycle observers, and a WebSocket constructor.

Related: fromEvent / share

`rxjs/webSocket` | API docs | Pinned source | ID: webSocket

02 / 8 entries

Join Creation Operators

Supply all sources directly. Tuple joins preserve positions; union joins forward values from different sources.

combineLatest

JOIN CREATION / CURRENT / COURSE LIST

```
combineLatest(sources: [A*, B*]): [A, B]*
```

After every source has supplied its first value, emit a tuple of the latest values whenever any source supplies another value.

- Subscription order matters for synchronous inputs: earlier sources can emit several times before the later sources are subscribed.
- A completed source keeps its last value available. The result completes when all sources have completed; any source error terminates the result.
- RxJS 7.8.2 still waits for all completions when a source completes empty. Therefore combineLatest([EMPTY, NEVER]) neither emits nor completes.
- With zero sources, complete without a next value.

```
combineLatest([of(1, 2), of('a', 'b')])
// next: [2, 'a'], [2, 'b']; complete
```

Scope. Shown for two Observable inputs. Any-length tuples, arrays, and keyed records are supported, and inputs may be ObservableInput. A record produces a record of latest values. Positional-source and scheduler-argument forms are deprecated.

Related: zip / forkJoin / withLatestFrom

`rxjs` | API docs | Pinned source | ID: combineLatest

concat

JOIN CREATION / CURRENT / COURSE LIST

```
concat(first: A*, second: B*): (A | B)*
```

Subscribe to the sources in sequence. Forward all values from the current source and move to the next only after it completes.

- A source that never completes prevents later sources from being subscribed.
- An error ends the sequence immediately; subsequent sources are not started.
- With no sources, complete without a next value. No scheduler is added by the common form.

```
concat(of(1, 2), of(3)) // next: 1, 2, 3; complete
```

Scope. Any number of ObservableInput sources can be passed as separate arguments. concat(...sources) joins an array of streams; concat(sources) instead treats that array as one input and emits its elements. A legacy scheduler overload is also available.

Related: concatAll / merge / onErrorResumeNext

`rxjs` | API docs | Pinned source | ID: concat

forkJoin

JOIN CREATION / CURRENT / COURSE LIST

```
forkJoin(sources: [A*, B*]): [A, B]*
```

Wait for all sources to complete, then emit one tuple containing each source's last value and complete. Every source must have supplied a value.

- If any source completes empty, the result completes without a next value and tears down active subscriptions.
- Otherwise, a source that never completes prevents the final tuple, even if it has already emitted.
- A source error terminates the result and tears down the other active inputs.
- With zero sources, complete without a next value.

```
forkJoin([of(1, 2), of('a', 'b')]) // next: [2, 'b']; complete
```

Scope. Shown for a pair of Observables. Tuples of any length, arrays, and keyed records are supported; actual inputs may be ObservableInput. Positional-source overloads are deprecated. The result emits at most one tuple in the selected form.

Related: combineLatest / zip / toArray

`rxjs` | API docs | Pinned source | ID: forkJoin

merge

JOIN CREATION / CURRENT / COURSE LIST

```
merge(first: A*, second: B*, concurrent?: number): (A | B)*
```

Forward values from active sources as they arrive. Several sources may be subscribed at the same time.

- Concurrency defaults to Infinity. A positive concurrency limit delays subscriptions to excess sources until active ones complete.
- Inputs are subscribed in argument order; synchronous earlier inputs can finish before later subscriptions begin.
- Complete when all sources finish. An error terminates the result and tears down active inputs; zero sources complete empty.

```
merge(of(1, 2), of('a')) // next: 1, 2, 'a'; complete
```

Scope. Shown for two sources; any number of ObservableInput sources can be supplied. Pass a trailing positive number to set concurrency. Scheduler-argument overloads are deprecated.

Related: mergeAll / concat / race

rxjs | API docs | Pinned source | ID: merge

partition

PARTITION / CURRENT / COURSE LIST

```
partition(source: A*, predicate: A => boolean): [A*, A*]
```

Return two streams: one passes values for which the predicate is true, and the other passes values for which it is false.

- This is a pair of Observables, not an Observable that emits pairs.
- Each branch subscribes to the input independently. Subscribing to both can run a cold source twice and evaluate the predicate separately.
- partition does not introduce sharing; arrange sharing explicitly if both branches must observe one source execution.

```
const [even$, odd$] = partition(of(1, 2, 3, 4), n => n % 2 === 0);
// even$: 2, 4, complete; odd$: 1, 3, complete (separate subscriptions)
```

Scope. The actual input may be ObservableInput, and predicates receive an index. A type-guard predicate narrows the true branch to B and the other branch to Exclude<A, B>. The standalone function is current; the old pipeable partition is deprecated.

Related: filter / share / partition-operator

rxjs | API docs | Pinned source | ID: partition

race

JOIN CREATION / CURRENT / COURSE LIST

```
race(sources: [A*, B*]): (A | B)*
```

The first next value selects the source whose subsequent values are forwarded. A completion or error before any next ends the result immediately.

- Once a source wins with a value, unsubscribe the other active candidates and follow the winner through its completion or error.
- Sources are subscribed in order. A synchronous winner or terminal notification can prevent later sources from ever being subscribed.
- Unlike most join creators, race([]) stays open without next values in RxJS 7.8.2.

```
race([of('first'), of('second')])
// next: 'first'; complete; the second source is never subscribed
```

Scope. Array and separate-argument forms accept any number of ObservableInput sources. The output payload is the union of source payloads.

Related: merge / timeout / takeUntil

`rxjs` | API docs | Pinned source | ID: race

zip

JOIN CREATION / CURRENT / COURSE LIST

```
zip(sources: [A*, B*]): [A, B]*
```

Match values by position: first with first, second with second, and so on. Emit a tuple only when every input has supplied its next unmatched value.

- Faster sources wait in per-source queues; this can retain many values when rates differ.
- Complete once a completed input has no buffered value left, because another full tuple is then impossible.
- A source error terminates the result and tears down the other inputs. With zero inputs, complete without next values.

```
zip([of(1, 2, 3), of('a', 'b')])
// next: [1, 'a'], [2, 'b']; complete; unmatched 3 is discarded
```

Scope. Any-length tuple and separate-argument forms accept ObservableInput sources. A result-selector overload maps each matched tuple to another payload type.

Related: combineLatest / forkJoin / zipWith

`rxjs` | API docs | Pinned source | ID: zip

onErrorResumeNext

JOIN CREATION / CURRENT / SUPPLEMENT

```
onErrorResumeNext(sources: [A*, B*]): (A | B)*
```

Run sources one after another, moving to the next after either completion or error. Values pass through; source errors do not.

- A normally completing source also advances to the next one; this is not a fallback that runs only on error.
- An error from a source is swallowed, so this operator cannot inspect it or choose recovery based on it.
- A source that never ends blocks the remaining sources. Once all sources have ended, complete; an empty list completes immediately.

```
onErrorResumeNext([of(1), throwError(() => new Error('skip')), of(2)])
// next: 1, 2; complete; the intermediate error is swallowed
```

Scope. Array and separate-argument forms accept ObservableInput sources. This standalone creator is current. Use onErrorResumeNextWith for the current pipeable form.

Related: concat / catchError / onErrorResumeNextWith

`rxjs` | API docs | Pinned source | ID: onErrorResumeNext

03 / 29 entries

Transformation Operators

Change payloads, accumulate state, group values, or choose an Action Policy for inner subscriptions.

buffer

PIPEABLE / CURRENT / COURSE LIST

```
buffer(closingNotifier: unknown*): A* => (A[])*
```

Collect source values into an array. Each closing notification emits the current array and starts an empty replacement.

- A closing next can emit an empty array; closing-notifier completion does not close the buffer or complete the result.
- Source completion emits the remaining array, including an empty one, then completes. Errors and unsubscription discard pending values.
- In RxJS 7.8.2 the source is subscribed before the closing notifier, which matters for synchronous sources.

```
of(1, 2, 3).pipe(buffer(NEVER)) // next: [1, 2, 3]; complete
```

Scope. The closing notifier can be any ObservableInput; the shorthand shows an Observable whose payload is ignored.

Related: bufferCount / bufferTime / bufferWhen / window

`rxjs` | API docs | Pinned source | ID: buffer

bufferCount

PIPEABLE / CURRENT / COURSE LIST

```
bufferCount(bufferSize: number, startBufferEvery?: number): A* => (A[])*
```

Collect values into arrays and emit each array when it reaches the chosen size.

- With positive counts, the default start interval is bufferSize, producing consecutive batches. A smaller interval overlaps buffers; a larger interval leaves gaps.
- Source completion emits all remaining partial buffers, then completes. An empty source produces no arrays.
- Buffers open when a source value arrives. Errors and unsubscription discard unfinished buffers.

```
of(1, 2, 3, 4, 5).pipe(bufferCount(2)) // next: [1, 2], [3, 4], [5]; complete
```

Scope. Passing null for startBufferEvery also selects bufferSize. The teaching form assumes positive buffer and start counts.

Related: buffer / pairwise / windowCount

rxjs | API docs | Pinned source | ID: bufferCount

bufferTime

PIPEABLE / CURRENT / COURSE LIST

```
bufferTime(timeSpan: number, scheduler?: SchedulerLike): A* => (A[])*
```

Collect values during each time span and emit the resulting array when that span ends.

- The first buffer opens on subscription. With the displayed form, each emitted buffer is immediately replaced; even empty buffers are emitted.
- The default clock is asyncScheduler and durations are in milliseconds. The operator schedules boundaries, while source values retain their arrival order within each buffer.
- Source completion flushes every active buffer immediately, then completes; it does not wait for their timers. Errors and unsubscription discard active buffers and cancel their timers.
- The longer forms can create overlapping buffers or gaps, and can close a buffer early at maxBufferSize. Without a creation interval, early closure immediately starts a replacement.

```
of(1, 2, 3).pipe(bufferTime(100)) // next: [1, 2, 3] immediately on source completion; complete
```

Scope. Also accepts (timeSpan, creationInterval, scheduler?) and (timeSpan, creationInterval, maxBufferSize, scheduler?). The creation interval defaults to no separate opening schedule and maxBufferSize to Infinity.

Related: buffer / bufferCount / windowTime

rxjs | API docs | Pinned source | ID: bufferTime

bufferToggle

PIPEABLE / CURRENT / COURSE LIST

```
bufferToggle(openings: O*, closingSelector: O => unknown*): A* => (A[])*
```

Each opening value starts a buffer and selects its closing notifier. That notifier's first next emits the buffer.

- Several buffers can be open together; a source value is added to all of them. Values arriving while no buffer is open are omitted.
- Closing-notifier completion alone leaves its buffer open. Completion of openings only prevents further openings.
- Source completion emits all still-open buffers, including empty ones, then completes. An error fails the result; unsubscription releases source and notifier subscriptions.
- Openings are subscribed before the source, so a synchronous opening can establish a buffer before a synchronous source starts.

```
of(1, 2, 3).pipe(bufferToggle(of('open'), () => NEVER)) // next: [1, 2, 3]; complete
```

Scope. Both openings and the closing-selector result can be ObservableInput values. The selector receives the opening payload, not a source payload.

Related: bufferWhen / bufferTime / windowToggle

`rxjs` | API docs | Pinned source | ID: bufferToggle

bufferWhen

PIPEABLE / CURRENT / COURSE LIST

```
bufferWhen(closingSelector: () => unknown*): A* => (A[])*
```

Open one buffer immediately. Each selected closing next emits that buffer, opens its replacement, and calls the selector again.

- Only one buffer is active at a time. The closing subscription is replaced after its first next; completion without a next leaves the current buffer open.
- Source completion emits the remaining buffer, including an empty array, then completes. Errors and unsubscription discard it.
- The first closing notifier is subscribed before the source. A selector that repeatedly returns an immediately emitting source can keep reopening buffers synchronously.

```
of(1, 2, 3).pipe(bufferWhen(() => NEVER)) // next: [1, 2, 3]; complete
```

Scope. The closing selector may return any ObservableInput. Unlike windowWhen, closing-notifier completion does not advance the buffer.

Related: buffer / bufferToggle / windowWhen

`rxjs` | API docs | Pinned source | ID: bufferWhen

concatMap

PIPEABLE / CURRENT / COURSE LIST

```
concatMap(project: A => B*): A* => B*
```

Process source values in order, one inner subscription at a time. Wait for the current inner to complete before starting the next.

- While an inner is active, source values queue. The projector is called when each queued value gets its turn, not when that value first arrives.
- Outer completion waits for the active inner and queued work. An inner that never completes blocks the queue.
- Errors terminate the result and tear down active subscriptions; downstream unsubscription also abandons queued work.
- The queue can grow when arrivals outpace completion. This is mergeMap with concurrency 1, without an added scheduling or sharing policy.

```
of(1, 2).pipe(concatMap(n => of(n, n * 10))) // next: 1, 10, 2, 20; complete
```

Scope. The projector also receives a zero-based processing index and can return any ObservableInput. The legacy resultSelector overload is deprecated; concatMap itself is current.

Related: concatAll / mergeMap / switchMap / exhaustMap

`rxjs` | API docs | Pinned source | ID: concatMap

concatMapTo

PIPEABLE / DEPRECATED / COURSE LIST

```
concatMapTo(inner: B*): A* => B*
```

For each source value, subscribe to the same supplied inner Observable, waiting for each subscription to complete before starting the next.

- Source payloads are ignored, but each source next reserves a turn in the queue.
- A cold inner starts its execution again for each subscription; a hot inner exposes whatever values arrive during that subscription.
- Outer completion waits for queued turns and the active inner. Errors or downstream unsubscription terminate the active work and abandon the queue.

Use now: concatMap(() => inner)

```
of('a', 'b').pipe(concatMapTo(of(1, 2))) // next: 1, 2, 1, 2; complete
```

Scope. The supplied inner may be any ObservableInput. The legacy resultSelector form also exists but is deprecated; the entire concatMapTo API is deprecated.

Related: concatMap / mergeMapTo / switchMapTo

`rxjs` | API docs | Pinned source | ID: concatMapTo

exhaust

PIPEABLE / DEPRECATED / COURSE LIST

```
exhaust(): (A*)* => A*
```

Subscribe to an arriving inner only while idle. Ignore arriving inners until the current one completes.

- This is the deprecated name of exhaustAll, with identical behavior.
- Ignored inners are neither subscribed nor queued. A synchronous inner can complete before the outer emits the next inner, allowing both to run.
- Outer completion waits for the accepted inner to complete. Errors and downstream unsubscription tear down active work.

Use now: exhaustAll()

```
of(timer(10), of(99)).pipe(exhaust()) // next: 0 after the timer; complete; of(99) is ignored
```

Scope. The outer can emit any ObservableInput; the shorthand shows an Observable of Observables.

Related: exhaustAll / exhaustMap / switchAll

`rxjs` | API docs | Pinned source | ID: exhaust

exhaustMap

PIPEABLE / CURRENT / COURSE LIST

```
exhaustMap(project: A => B*): A* => B*
```

While idle, project a source value and subscribe to its inner. Ignore further source values until that inner completes.

- Ignored values are not queued and do not call the projector. Only accepted values advance the projector's index.
- A never-completing inner keeps the operator busy indefinitely. A synchronous inner may finish before the next source value arrives.
- Outer completion waits for the active inner. Source, projection, or inner errors fail the result; downstream unsubscription tears down active work.

```
of(1, 2, 3).pipe(exhaustMap(n => of(n).pipe(delay(10)))) // next: 1; complete; 2 and 3 are ignored
while busy
```

Scope. The projector also receives a zero-based accepted-value index and may return any ObservableInput. Only the legacy resultSelector overload is deprecated.

Related: exhaustAll / concatMap / mergeMap / switchMap

`rxjs` | API docs | Pinned source | ID: exhaustMap

expand

PIPEABLE / CURRENT / COURSE LIST

```
expand(project: A => A*, concurrent?: number): A* => A*
```

Emit each source value and recursively project every value reached. Values produced by inner streams become further inputs to the same projection.

- The starting source values are included in the output. Returning EMPTY ends that branch without removing the value that led to it.
- The default concurrency is Infinity. A finite positive limit queues values until an inner subscription slot becomes available; projection and output order depend on that limit and inner timing.
- Completion requires the source, all recursive inner work, and the queue to finish. Errors or downstream unsubscription tear down active subscriptions.
- There is no built-in visited set or cycle detection. A projection that keeps producing values can continue indefinitely.

```
of(1).pipe(expand(n => n < 4 ? of(n + 1) : EMPTY)) // next: 1, 2, 3, 4; complete
```

Scope. Shows recursion within one payload type A. The projector also receives an index and may return ObservableInput. The legacy scheduler argument is deprecated; schedule inner subscriptions with subscribeOn when needed.

Related: mergeMap / mergeScan / take

rxjs | API docs | Pinned source | ID: expand

groupBy

PIPEABLE / CURRENT / COURSE LIST

```
groupBy(key: A => K): A* => (A* & { readonly key: K })*
```

Route each value into the stream for its key. Emit a new keyed stream when that key first appears, then send matching values through that stream.

- A group is a GroupedObservable: an Observable with a key property. Default groups use Subjects, are hot, and do not replay missed values; subscribe when the group is emitted.
- Without a duration selector, groups remain tracked until the source terminates. A duration next or completion expires that group; a later value with the same key creates a new group.
- Source, key-selector, and element-selector errors reach the outer stream and all groups. A duration-stream error affects its own group; source completion completes all current groups and the outer stream.
- Subscribed groups can keep the source subscription alive after the outer group stream is unsubscribed. Teardown waits for the remaining group subscriptions to end.

```
of(1, 2, 3, 4).pipe(groupBy(n => n % 2), mergeMap(group => group.pipe(toArray(), map(values => ({
key: group.key, values })))))) // next: { key: 1, values: [1, 3] }, { key: 0, values: [2, 4] };
complete
```

Scope. The options form can select element payloads, group duration, and a custom connector. With element: A => B, the result is (B* & { readonly key: K })*. A type-guard key has narrowing overloads. Legacy positional element/duration/connector overloads are deprecated; groupBy itself is current.

Related: partition / window / mergeMap

rxjs | API docs | Pinned source | ID: groupBy

map

PIPEABLE / CURRENT / COURSE LIST

```
map(project: A => B): A* => B*
```

Apply the projection to each source value and emit its returned value.

- Projection runs during each source next; map adds no scheduling step.
- If the projection returns an Observable, that Observable is the payload. It is not automatically subscribed or flattened: the resulting shape is (B*)*.
- A thrown projection error fails the result. Otherwise, source completion and errors pass through.

```
of(1, 2, 3).pipe(map(n => n * 10)) // next: 10, 20, 30; complete
```

Scope. The projector also receives a zero-based source index. The thisArg overload is deprecated; map itself is current.

Related: filter / scan / switchMap / mapTo

rxjs | API docs | Pinned source | ID: map

mapTo

PIPEABLE / DEPRECATED / COURSE LIST

```
mapTo(value: B): A* => B*
```

Replace every source value with the same supplied value.

- Source next notifications determine when the constant is emitted; source payloads are ignored.
- The supplied value is reused. An object is the same reference on every emission, and an Observable value is emitted without being subscribed.
- An empty source produces no constant value. Source completion and errors pass through.

Use now: map(() => value)

```
of(1, 2).pipe(mapTo('saved')) // next: 'saved', 'saved'; complete
```

Scope. The public input type is unknown because the source payload is unused.

Related: map / startWith

rxjs | API docs | Pinned source | ID: mapTo

mergeMap

PIPEABLE / CURRENT / COURSE LIST

```
mergeMap(project: A => B*, concurrent?: number): A* => B*
```

Project source values into inner streams and forward their values as they arrive. Several inner subscriptions may be active together.

- Concurrency defaults to Infinity. With a finite positive limit, excess source values queue and the projector is called only when a subscription slot opens.
- Output follows inner emission order, which can differ from source order. With concurrency 1, the processing policy is concatMap.
- Outer completion waits for active inners and queued work. Errors fail the result and tear down active subscriptions; downstream unsubscription also abandons queued values.
- A concurrency limit bounds active inners, not the queue. Queued source values can accumulate when arrivals outpace inner completion.

```
of(1, 2).pipe(mergeMap(n => of(n).pipe(delay(n === 1 ? 20 : 10)))) // next: 2, 1 with these delays; complete
```

Scope. The projector also receives a zero-based processing index and may return any ObservableInput. The legacy resultSelector overload is deprecated; mergeMap itself is current.

Related: mergeAll / concatMap / switchMap / exhaustMap

rxjs | API docs | Pinned source | ID: mergeMap

mergeMapTo

PIPEABLE / DEPRECATED / COURSE LIST

```
mergeMapTo(inner: B*, concurrent?: number): A* => B*
```

For each source value, subscribe to the same supplied inner and merge the values from those subscriptions.

- Source payloads are ignored. Concurrency defaults to Infinity; a finite positive limit queues source triggers until a slot is available.
- Each trigger creates another subscription. A cold inner can repeat its work, while multiple subscriptions to a hot inner can forward the same event more than once.
- Outer completion waits for active and queued subscriptions. Errors or downstream unsubscription end active work and abandon the queue.

Use now: mergeMap(() => inner, concurrent)

```
of('a', 'b').pipe(mergeMapTo(of(1, 2))) // next: 1, 2, 1, 2; complete
```

Scope. The supplied inner may be any ObservableInput. The legacy resultSelector form is also deprecated.

Related: mergeMap / concatMapTo / switchMapTo

rxjs | API docs | Pinned source | ID: mergeMapTo

mergeScan

PIPEABLE / CURRENT / COURSE LIST

```
mergeScan(accumulator: (S, A) => S*, seed: S, concurrent?: number): A* => S*
```

Use the current state and an input to start an inner stream of states. Merge all active inners, remembering each emitted state for the next accumulator call.

- The seed initializes state but is not emitted. The accumulator sees the latest inner value emitted so far, or the seed if no inner has emitted.
- Concurrency defaults to Infinity. Overlapping accumulations can start from the same state, and a later emission from an older inner can replace a newer state.
- With concurrency 1, queued inputs call the accumulator after prior inners complete, using the latest state at that moment. This serializes asynchronous state transitions.
- Outer completion waits for inners and queued work. State belongs to each subscription, but a supplied object seed is not cloned.

```
of(1, 2, 3).pipe(mergeScan((state, n) => of(state + n), 0)) // next: 1, 3, 6; complete
```

Scope. The accumulator also receives a zero-based processing index and may return ObservableInput<S>. There is no unseeded overload.

Related: scan / switchScan / mergeMap / expand

rxjs | API docs | Pinned source | ID: mergeScan

pairwise

PIPEABLE / CURRENT / COURSE LIST

```
pairwise(): A* => [A, A]*
```

Starting with the second source value, emit the previous and current values as a tuple.

- The tuple order is [previous, current]. Adjacent tuples overlap in one value.
- A source with zero or one value emits no pair. There is no final incomplete tuple when the source completes.
- Only the previous value is retained between notifications; each pair is a new array, while its element references are unchanged.

```
of(1, 2, 3).pipe(pairwise()) // next: [1, 2], [2, 3]; complete
```

Related: bufferCount / scan

rxjs | API docs | Pinned source | ID: pairwise

partition

PARTITION / DEPRECATED / COURSE LIST / partition-operator

```
partition(predicate: A => boolean): A* => [A*, A*]
```

Return two streams: values that pass the predicate and values that fail it.

- This legacy rxjs/operators form returns an array of streams, not an Observable emitting arrays. It ends the ordinary Observable pipeline at that point.
- Each branch subscribes to the source separately and runs its own predicate evaluation. A cold source can execute twice when both branches are subscribed.
- Sharing is a separate decision. With a hot source, each branch observes only values available during its own subscription.

Use now: partition(source, predicate) imported from rxjs

```
of(1, 2, 3).pipe(partition(n => n % 2 === 0)) // returns [passing$, failing$]; subscribing yields 2 / 1, 3; each completes
```

Scope. Import this legacy form from rxjs/operators. The predicate also receives a zero-based index and an optional thisArg is supported. Prefer the current static partition from rxjs.

Related: partition / filter / groupBy / share

`rxjs/operators` | API docs | Pinned source | ID: partition-operator

pluck

PIPEABLE / DEPRECATED / COURSE LIST

```
pluck(key: K): A* => (A[K])*
```

Read a property from every source value and emit the selected property value.

- A nested path walks successive properties. If the path cannot be resolved, the operator emits undefined rather than omitting that input.
- A selected null value is still an emitted value. Getter exceptions fail the result, just as a thrown map projection would.
- Calling pluck with no property names throws when the operator is created.

Use now: map(value => value?.property), with optional chaining for nested paths

```
of({ name: 'Ada' }, { name: 'Erik' }).pipe(pluck('name')) // next: 'Ada', 'Erik'; complete
```

Scope. Shows one known key, K extends keyof A, with the indexed property type A[K]. Typed overloads support nested paths; a general string path returns unknown. Real runtime data with missing properties can produce undefined.

Related: map

`rxjs` | API docs | Pinned source | ID: pluck

scan

PIPEABLE / CURRENT / COURSE LIST

```
scan(reducer: (S, A) => S, seed: S): A* => S*
```

Update the current state with each source value and emit that updated state.

- The seed supplies the first state but is not emitted on subscription. An empty source produces no state values.
- Each subscription tracks its own state variable. A supplied object seed is reused by reference, so pure reducers should return the intended next state without mutating shared seed data.
- Reduction runs when each input arrives. A thrown reducer error fails the result; source completion does not emit an extra final state.

```
of(1, 2, 3).pipe(scan((state, n) => state + n, 0)) // next: 1, 3, 6; complete
```

Scope. Shows the seeded form with one state type S. Without a seed, the first source value becomes state and is emitted without calling the reducer. Other overloads distinguish initial and accumulated state types. The reducer also receives the source index; explicitly passing undefined still supplies a seed.

Related: reduce / mergeScan / switchScan / startWith

rxjs | API docs | Pinned source | ID: scan

switchScan

PIPEABLE / CURRENT / COURSE LIST

```
switchScan(accumulator: (S, A) => S*, seed: S): A* => S*
```

Use the current state and each input to select an inner stream of states. Replace the previous inner, forwarding and remembering values only from the active one.

- The seed is not emitted. State is the latest inner value already emitted, so a canceled inner that emitted nothing has not advanced state.
- On each new input, the prior inner is unsubscribed before the new accumulator call. Values already emitted by that inner remain part of the observed history.
- Outer completion waits for the current inner. Errors or downstream unsubscription tear down active work.
- State tracking is per subscription, but the supplied seed object is not cloned. The underlying work stops on cancellation only when the inner's teardown stops it.

```
of(1, 2).pipe(switchScan((state, n) => of(state + n).pipe(delay(10)), 0)) // next: 2; complete; the first inner never emits
```

Scope. Shows one state type S. The accumulator also receives the source index and may return ObservableInput. There is no unseeded overload.

Related: scan / mergeScan / switchMap

rxjs | API docs | Pinned source | ID: switchScan

switchMap

PIPEABLE / CURRENT / COURSE LIST

```
switchMap(project: A => B*): A* => B*
```

For each input, replace the current inner subscription with the projected inner. Forward values from the active inner only.

- The previous inner is unsubscribed before the next projector call and subscription. Previously emitted values are not undone.
- Outer completion waits for the current inner to complete; an outer next, not outer completion, triggers replacement.
- Unsubscribing stops observing the old inner. Stopping its underlying request, timer, or other work depends on that inner's teardown.
- Inner streams determine emission timing. switchMap adds no sharing, and source, projection, or inner errors fail the result.

```
of(1, 2).pipe(switchMap(n => of(n).pipe(delay(10)))) // next: 2; complete; the first inner is canceled
```

Scope. The projector also receives the source index and may return any ObservableInput. The legacy resultSelector overload is deprecated; switchMap itself is current.

Related: switchAll / map / concatMap / mergeMap / exhaustMap

`rxjs` | API docs | Pinned source | ID: switchMap

switchMapTo

PIPEABLE / DEPRECATED / COURSE LIST

```
switchMapTo(inner: B*): A* => B*
```

Each source value cancels the current inner subscription and starts another subscription to the same supplied inner.

- Source payloads are ignored. A cold inner starts again; a hot inner supplies the events available to its replacement subscription.
- Cancellation does not undo earlier emissions, and stopping underlying work depends on the inner's teardown.
- Outer completion waits for the current inner. Errors or downstream unsubscription terminate active work.

Use now: switchMap(() => inner)

```
of(1, 2).pipe(switchMapTo(timer(10))) // next: 0 from the replacement timer; complete
```

Scope. The supplied inner may be any ObservableInput. The legacy resultSelector form is also deprecated.

Related: switchMap / concatMapTo / mergeMapTo

`rxjs` | API docs | Pinned source | ID: switchMapTo

window

PIPEABLE / CURRENT / COURSE LIST

```
window(boundaries: unknown*): A* => (A*)*
```

Emit a live window stream immediately. Each boundary next completes that window and emits a new one.

- Windows are hot and do not replay past values. Subscribe to each window when it is emitted, commonly by immediately flattening or processing it.
- Boundary completion alone does not close the window or complete the result. Source completion completes the current window and the outer stream.
- Source or boundary errors error both the current window and the outer stream. Downstream unsubscription tears down the source and boundary subscriptions.
- The initial window is emitted before source subscription; the boundary subscription is established after the source subscription in RxJS 7.8.2.

```
of(1, 2, 3).pipe(window(NEVER), mergeMap(w => w.pipe(toArray()))) // next: [1, 2, 3]; complete
```

Scope. The boundary source may be any ObservableInput. Its next payload is ignored; it supplies window boundaries only.

Related: buffer / windowCount / windowTime / windowWhen

rxjs | API docs | Pinned source | ID: window

windowCount

PIPEABLE / CURRENT / COURSE LIST

```
windowCount(windowSize: number, startWindowEvery?: number): A* => (A*)*
```

Divide source values into live window streams, completing each window after its chosen number of values.

- For positive counts, the default start interval equals windowSize. Smaller intervals overlap windows; larger intervals leave values outside any window.
- Windows are hot and need timely subscriptions. Each window is emitted when it opens, before its values arrive.
- RxJS 7.8.2 opens the initial window immediately and eagerly opens each next window after the start interval. An empty source or an exact count boundary can therefore leave an empty window.
- Source completion completes all open windows and the outer stream; source errors error them. Window completion does not wait for consumers to subscribe.

```
of(1, 2, 3, 4).pipe(windowCount(2), mergeMap(w => w.pipe(toArray()))) // next: [1, 2], [3, 4], [];
complete
```

Scope. The displayed behavior assumes positive windowSize. An omitted or nonpositive startWindowEvery uses windowSize.

Related: bufferCount / window / windowTime

rxjs | API docs | Pinned source | ID: windowCount

windowTime

PIPEABLE / CURRENT / COURSE LIST

```
windowTime(timeSpan: number, scheduler?: SchedulerLike): A* => (A*)*
```

Emit a live window for each time span. In the displayed form, completing one timed window immediately opens the next.

- A window is emitted when it opens, starting immediately on subscription. It is hot and does not replay missed source values.
- The default scheduler is asyncScheduler, with millisecond durations. Timers close windows; source arrivals supply their values, and windows can be empty.
- Source completion completes active windows immediately and completes the outer stream. Source errors error both; unsubscription cancels source and scheduled boundary work.
- Longer forms support overlapping windows, gaps, and a maximum value count. Without a separate creation interval, hitting the count limit immediately opens a replacement.

```
of(1, 2, 3).pipe(windowTime(100), mergeMap(w => w.pipe(toArray()))) // next: [1, 2, 3] immediately on source completion; complete
```

Scope. Also accepts (timeSpan, creationInterval, scheduler?) and (timeSpan, creationInterval, maxWindowSize, scheduler?). The creation interval defaults to no separate opening schedule and maxWindowSize to Infinity.

Related: bufferTime / windowCount / windowToggle

rxjs | API docs | Pinned source | ID: windowTime

windowToggle

PIPEABLE / CURRENT / COURSE LIST

```
windowToggle(openings: O*, closingSelector: O => unknown*): A* => (A*)*
```

Each opening value creates and emits a live window. Its selected closing notifier completes that window on the first next.

- Windows can overlap or leave gaps. A source value is sent to every open window; with no open window it is omitted.
- Closing-notifier completion alone does not close a window. Completion of openings prevents new openings without completing current windows.
- Windows are hot and require timely subscriptions. Openings are subscribed before the source, allowing synchronous openings to establish windows first.
- Source completion completes all current windows and the outer stream. Errors fail the outer stream; downstream unsubscription tears down source and notifier subscriptions.

```
of(1, 2, 3).pipe(windowToggle(of('open'), () => NEVER), mergeMap(w => w.pipe(toArray()))) // next: [1, 2, 3]; complete
```

Scope. Openings and closing-selector results may be ObservableInput values. The selector receives the opening value rather than the source value.

Related: bufferToggle / windowWhen / windowTime

rxjs | API docs | Pinned source | ID: windowToggle

windowWhen

PIPEABLE / CURRENT / COURSE LIST

```
windowWhen(closingSelector: () => unknown*): A* => (A*)*
```

Open and emit a window immediately. A selected closing next or completion closes it, opens a replacement, and calls the selector again.

- Only one window is active at a time. Unlike bufferWhen, a closing notifier that completes without a value also advances the window.
- Windows are hot and do not replay past values. Subscribe when each is emitted to observe its contents.
- Source completion completes the current window and outer stream. Source, selector, or closing errors error the window and the result; unsubscription tears down the active subscriptions.
- The first closing notifier is subscribed before the source. Repeated immediately emitting or completing closers can reopen windows synchronously without letting the source run.

```
of(1, 2, 3).pipe(windowWhen(() => NEVER), mergeMap(w => w.pipe(toArray()))) // next: [1, 2, 3]; complete
```

Scope. The closing selector may return any ObservableInput.

Related: bufferWhen / window / windowToggle

`rxjs` | API docs | Pinned source | ID: windowWhen

flatMap

PIPEABLE / DEPRECATED / SUPPLEMENT

```
flatMap(project: A => B*, concurrent?: number): A* => B*
```

Project values to inner streams and merge their emissions. This is the deprecated alias of mergeMap.

- Concurrency defaults to Infinity. A finite positive limit queues source values until an inner subscription can start.
- Output follows inner emission timing, so source order is not guaranteed. Outer completion waits for active and queued work.
- Errors or downstream unsubscription end active subscriptions and abandon queued work, exactly as with mergeMap.

Use now: mergeMap(project, concurrent)

```
of(1, 2).pipe(flatMap(n => of(n, n * 10))) // next: 1, 10, 2, 20; complete
```

Scope. Inherits mergeMap's ObservableInput projector, processing index, concurrency option, and legacy deprecated resultSelector overloads.

Related: mergeMap / mergeAll

`rxjs` | API docs | Pinned source | ID: flatMap

04 / 25 entries

Filtering Operators

Choose which values pass. Similar A* => A* shapes can have very different gates, deadlines, and completion rules.

audit

PIPEABLE / CURRENT / COURSE LIST

```
audit(durationSelector: A => unknown*): A* => A*
```

The first value starts a duration. While that duration is active, remember the latest source value; emit it when the duration sends a next notification.

- New source values replace the pending value without restarting the active duration.
- Duration completion without a next notification closes the duration without emitting the pending value.
- Source completion waits for an active duration: a duration next releases the pending value before completion, while a duration completion without next discards it.
- A source or duration error terminates the result and discards the pending value.

```
of(1, 2, 3).pipe(audit(() => timer(10))) // next: 3 when the duration signals; complete
```

Scope. The duration selector also accepts other ObservableInput values, including promises. Its emitted payload is ignored.

Related: auditTime / debounce / throttle / sample

rxjs | API docs | Pinned source | ID: audit

auditTime

PIPEABLE / CURRENT / COURSE LIST

```
auditTime(duration: number, scheduler?: SchedulerLike): A* => A*
```

The first value opens a timed interval. Emit the latest value at the end of that interval, then wait for another source value to open the next interval.

- Values arriving during an interval replace the pending value without moving its deadline.
- Intervals are started by source values; there is no continuously running sampling clock.
- Source completion waits for an active interval and its pending emission. An error or downstream unsubscription discards that pending value.
- The scheduler defaults to asyncScheduler; duration is measured in its time units, normally milliseconds.

```
of(1, 2, 3).pipe(auditTime(10)) // next: 3 after the interval; complete
```

Scope. Equivalent to audit(() => timer(duration, scheduler)).

Related: audit / debounceTime / sampleTime / throttleTime

rxjs | API docs | Pinned source | ID: auditTime

debounce

PIPEABLE / CURRENT / COURSE LIST

```
debounce(durationSelector: A => unknown*): A* => A*
```

Each source value replaces the pending value and starts a new duration. Emit that value if its duration sends a next notification before another source value arrives.

- Every new source value unsubscribes the previous duration and chooses a replacement duration.
- Duration completion without next does not release the pending value.
- Source completion immediately emits any pending value and then completes, even if the duration has not signaled.
- An error or downstream unsubscription discards the pending value.

```
of(1, 2, 3).pipe(debounce(() => timer(10))) // next: 3 immediately on source completion; complete
```

Scope. The duration selector also accepts other ObservableInput values. The duration payload is ignored.

Related: debounceTime / audit / switchMap

rxjs | API docs | Pinned source | ID: debounce

debounceTime

PIPEABLE / CURRENT / COURSE LIST

```
debounceTime(dueTime: number, scheduler?: SchedulerLike): A* => A*
```

Wait for a quiet period after a source value, then emit the latest pending value. Each new value moves the deadline.

- A continuously busy source can keep replacing the pending value without producing output.
- Source completion immediately releases the pending value and then completes; it does not wait out the remaining delay.
- A source error or downstream unsubscription discards the pending value.
- The scheduler defaults to asyncScheduler; dueTime uses its time units, normally milliseconds.

```
of(1, 2, 3).pipe(debounceTime(10)) // next: 3 immediately on source completion; complete
```

Scope. The output remains A*: the operator can emit many values across separate quiet periods.

Related: debounce / auditTime / throttleTime / sampleTime

rxjs | API docs | Pinned source | ID: debounceTime

distinct

PIPEABLE / CURRENT / COURSE LIST

```
distinct(keySelector?: A => K, flushes?: unknown*): A* => A*
```

Emit a source value only when its selected key has not been seen in this subscription since the last flush.

- Without a key selector, the source value itself is the key. Keys are stored in a JavaScript Set: objects compare by identity and repeated NaN keys count as duplicates.
- A flushes next notification clears the remembered keys; flushes completion alone does not clear them or complete the result.
- The remembered set can grow for the lifetime of a subscription; supplying a key selector does not impose a size limit.

```
of(1, 1, 2, 1, 3, 2).pipe(distinct()) // next: 1, 2, 3; complete
```

Scope. K is the selected key type. The optional flushes input also accepts other ObservableInput values.

Related: distinctUntilChanged / distinctUntilKeyChanged / filter

rxjs | API docs | Pinned source | ID: distinct

distinctUntilChanged

PIPEABLE / CURRENT / COURSE LIST

```
distinctUntilChanged(compare?: (A, A) => boolean): A* => A*
```

Emit the first value, then emit a later value only when the comparator says it differs from the last emitted value.

- Comparator true suppresses the new value; comparator false emits it and updates the remembered value.
- The default comparator is ===. It does not perform a deep object comparison.
- Suppressed values do not replace the comparison baseline; only the last emitted value is remembered.

```
of(1, 1, 2, 2, 1).pipe(distinctUntilChanged()) // next: 1, 2, 1; complete
```

Scope. The key-selector form is distinctUntilChanged(compare: (K, K) => boolean, keySelector: A => K): A* => A*. It compares selected keys but emits the original A values.

Related: distinct / distinctUntilKeyChanged / pairwise

rxjs | API docs | Pinned source | ID: distinctUntilChanged

distinctUntilKeyChanged

PIPEABLE / CURRENT / COURSE LIST

```
distinctUntilKeyChanged(key: K, compare?: (A[K], A[K]) => boolean): A* => A*
```

Emit the first object, then suppress objects whose chosen property compares equal to that property on the last emitted object.

- The default comparison is ===; a custom comparator returning true suppresses the new object.
- The entire object is emitted, including properties that were not compared.
- The previous object is retained by reference, so mutating it can change a later comparison.

```
of({ id: 1 }, { id: 1 }, { id: 2 }).pipe(distinctUntilKeyChanged('id')) // next: { id: 1 }, { id: 2 }; complete
```

Scope. K extends keyof A; A[K] is the type of the selected property.

Related: distinctUntilChanged / distinct / filter

rxjs | API docs | Pinned source | ID: distinctUntilKeyChanged

elementAt

PIPEABLE / CURRENT / COURSE LIST

```
elementAt(index: number): A* => A*
```

Emit the value at a zero-based position, then complete and unsubscribe from the source.

- At most one value is emitted. Counting begins with the first value observed by this subscription.
- If the source completes before that position exists, the selected form errors with ArgumentOutOfRangeError.
- A negative index throws when elementAt is called, before subscription.

```
of('a', 'b', 'c').pipe(elementAt(1)) // next: 'b'; complete
```

Scope. Use a non-negative integer index. Providing defaultValue: D changes the output to (A | D)* and emits the default on normal completion if the position was not reached.

Related: first / take / skip / single

rxjs | API docs | Pinned source | ID: elementAt

filter

PIPEABLE / CURRENT / COURSE LIST

```
filter(predicate: A => boolean): A* => A*
```

Keep each source value whose predicate returns true. Other values contribute no output.

- The predicate is evaluated independently for each source value; a rejection does not stop later values from being tested.
- Values that pass are forwarded during the source notification; filter adds no scheduling step.
- The full result is A*: an individual input can contribute zero or one value, but the subscription can emit many values.

```
of(1, 2, 3, 4).pipe(filter(n => n % 2 === 0)) // next: 2, 4; complete
```

Scope. The callback also receives a zero-based index. A type guard (value: A) => value is B, with B extends A, produces B*. Boolean has a truthiness-narrowing overload. Only overloads accepting thisArg are deprecated.

Related: map / takeWhile / skipWhile / partition

rxjs | API docs | Pinned source | ID: filter

first

PIPEABLE / CURRENT / COURSE LIST

```
first(predicate?: A => boolean): A* => A*
```

Emit the first matching value and complete immediately. Without a predicate, the first source value matches.

- Once a match is emitted, the source subscription is closed; later values are not needed.
- If the source completes without a match, the selected form errors with EmptyError.
- An input that never matches and never completes can keep the result open without emitting.

```
of(1, 2, 3).pipe(first(n => n > 1)) // next: 2; complete
```

Scope. A second argument defaultValue: D changes the output to (A | D)* and replaces the no-match error on completion. Use first(null, defaultValue) without a predicate. Type-guard and Boolean overloads can narrow the payload; predicates also receive index and source.

Related: take / find / last / single / elementAt

rxjs | API docs | Pinned source | ID: first

ignoreElements

PIPEABLE / CURRENT / COURSE LIST

```
ignoreElements(): A* => never*
```

Suppress every next value while preserving the source's completion or error notification.

- The source is still subscribed and executes normally; ignoring its values does not prevent its side effects.
- A source that never terminates leaves the result open. never* describes the absence of next values, not the termination behavior.

```
of(1, 2, 3).pipe(ignoreElements()) // no next values; complete
```

Scope. The native signature accepts unknown payloads and produces Observable<never>.

Related: filter / tap / materialize / endWith

rxjs | API docs | Pinned source | ID: ignoreElements

last

PIPEABLE / CURRENT / COURSE LIST

```
last(predicate?: A => boolean): A* => A*
```

Remember the latest matching value and emit it when the source completes. Without a predicate, every source value matches.

- It must wait for normal source completion to know which matching value was last.
- If the source completes without a match, the selected form errors with EmptyError.
- A source error or downstream unsubscription discards the remembered value.

```
of(1, 2, 3, 4).pipe(last(n => n % 2 === 1)) // next: 3 at source completion; complete
```

Scope. A second argument defaultValue: D changes the output to (A | D)* and replaces the no-match error on completion. Use last(null, defaultValue) without a predicate. Type-guard and Boolean overloads can narrow the payload; predicates also receive index and source.

Related: first / takeLast / single / reduce

rxjs | API docs | Pinned source | ID: last

sample

PIPEABLE / CURRENT / COURSE LIST

```
sample(notifier: unknown*): A* => A*
```

On each notifier next, emit the latest source value received since the previous sample. If no new source value arrived, emit nothing.

- Sampling clears the pending-value flag, so repeated notifier values do not replay the same source value.
- Notifier completion is ignored. Source completion immediately completes the result without flushing a pending value.
- The source is subscribed before the notifier; a synchronous source can finish before the notifier is subscribed.
- A notifier error terminates the result, as does a source error.

```
If the source emits 1 then 2 before a notifier next, that next releases 2. A second notifier next
with no new source value emits nothing. Source completion then completes the result.
```

Scope. The notifier also accepts other ObservableInput values. Its emitted payload is ignored.

Related: sampleTime / audit / withLatestFrom

rxjs | API docs | Pinned source | ID: sample

sampleTime

PIPEABLE / CURRENT / COURSE LIST

```
sampleTime(period: number, scheduler?: SchedulerLike): A* => A*
```

At regular clock ticks, emit the latest source value received since the previous tick. A tick with no new value produces no output.

- The sampling clock runs independently of source arrivals; arrivals do not move its next tick.
- Source completion immediately completes the result and discards any value waiting for a later tick.
- The scheduler defaults to asyncScheduler; period uses its time units, normally milliseconds.

```
With a 10 ms period, values 1 at 2 ms and 2 at 7 ms produce 2 at the 10 ms tick. If the source completes at 15 ms, the result completes then.
```

Scope. Equivalent to sample(interval(period, scheduler)).

Related: sample / auditTime / debounceTime / throttleTime

rxjs | API docs | Pinned source | ID: sampleTime

single

PIPEABLE / CURRENT / COURSE LIST

```
single(predicate?: A => boolean): A* => A*
```

Require exactly one matching value across the source. Emit that value only after source completion proves there was no second match.

- A second match causes an immediate SequenceError, without emitting the first match.
- A source that completes without any values causes EmptyError.
- A nonempty source that completes with no predicate match causes NotFoundError.
- There is no default-value argument. A nonterminating source with only one match still cannot establish uniqueness.

```
of(1, 2, 3).pipe(single(n => n % 2 === 0)) // next: 2 at source completion; complete
```

Scope. Without a predicate, all source values count as matches. Predicates also receive index and source. A Boolean overload narrows truthy payloads.

Related: first / last / find / elementAt

rxjs | API docs | Pinned source | ID: single

skip

PIPEABLE / CURRENT / COURSE LIST

```
skip(count: number): A* => A*
```

Discard the first count source values, then forward later values.

- Counting is local to each subscription and does not depend on elapsed time.
- A source that completes before enough values arrive simply completes without output.
- For count zero or below, values pass through.

```
of(1, 2, 3, 4).pipe(skip(2)) // next: 3, 4; complete
```

Scope. The teaching form assumes an integer count.

Related: take / skipWhile / skipUntil / skipLast

rxjs | API docs | Pinned source | ID: skip

skipLast

PIPEABLE / CURRENT / COURSE LIST

```
skipLast(count: number): A* => A*
```

Hold back the latest count values. Each later arrival releases the oldest held value; the final held values are discarded when the source ends.

- This introduces a lag measured in source values, not a timed delay.
- Normal completion does not flush the held values. A source with at most count values completes without emitting.
- Positive integer counts retain up to count values; zero or negative counts pass values through.

```
of(1, 2, 3, 4).pipe(skipLast(2)) // next: 1 when 3 arrives, 2 when 4 arrives; complete
```

Scope. The teaching form assumes an integer count.

Related: takeLast / skip / delay

rxjs | API docs | Pinned source | ID: skipLast

skipUntil

PIPEABLE / CURRENT / COURSE LIST

```
skipUntil(notifier: unknown*): A* => A*
```

Discard source values until the notifier sends its first next notification. Forward later source values from then on.

- The notifier is subscribed before the source, so a synchronous notifier next can open the gate before the first source value.
- After opening the gate, the notifier is unsubscribed; it cannot close the gate again.
- Notifier completion without next leaves the gate closed. Source completion still completes the result; notifier errors terminate it.

```
of(1, 2, 3).pipe(skipUntil(of('go'))) // next: 1, 2, 3; complete
```

Scope. The notifier also accepts other ObservableInput values; its payload is ignored.

Related: takeUntil / skipWhile / skip

rxjs | API docs | Pinned source | ID: skipUntil

skipWhile

PIPEABLE / CURRENT / COURSE LIST

```
skipWhile(predicate: A => boolean): A* => A*
```

Discard the initial run of values for which the predicate is true. Emit the first value for which it is false and every later value.

- The predicate stops being called after its first false result.
- Later values pass through even if they would have satisfied the original skipping condition.
- If the predicate stays true until source completion, the result completes without values.

```
of(1, 2, 3, 1).pipe(skipWhile(n => n < 3)) // next: 3, 1; complete
```

Scope. The callback also receives a zero-based index. Native overloads include Boolean and an always-true predicate; skipWhile does not generally provide type-guard narrowing for later values.

Related: takeWhile / filter / skipUntil

rxjs | API docs | Pinned source | ID: skipWhile

take

PIPEABLE / CURRENT / COURSE LIST

```
take(count: number): A* => A*
```

Forward up to count source values, then complete and unsubscribe from the source.

- A positive count is an upper bound, not a promise that the source will supply that many values.
- Early source completion is forwarded without an empty-source error.
- For count zero or below, the result completes immediately without subscribing to the source.

```
of(1, 2, 3).pipe(take(2)) // next: 1, 2; complete
```

Scope. The teaching form assumes an integer count. take(1) permits zero values; first() instead errors if its source completes empty.

Related: first / takeUntil / takeWhile / skip

rxjs | API docs | Pinned source | ID: take

takeLast

PIPEABLE / CURRENT / COURSE LIST

```
takeLast(count: number): A* => A*
```

Remember up to count recent values and emit them in their original order when the source completes.

- For a positive count, no values are emitted before normal source completion.
- If the source supplies fewer than count values and completes, all its values are emitted; an empty source completes without error.
- A source error or downstream unsubscription discards the buffered values.
- For count zero or below, the result completes immediately without subscribing to the source.

```
of(1, 2, 3, 4).pipe(takeLast(2)) // next: 3, 4 at source completion; complete
```

Scope. The teaching form assumes an integer count. The buffer retains at most count values for a positive integer count.

Related: last / skipLast / toArray / take

rxjs | API docs | Pinned source | ID: takeLast

takeUntil

PIPEABLE / CURRENT / COURSE LIST

```
takeUntil(notifier: unknown*): A* => A*
```

Forward source values until the notifier sends a next notification. That notification completes the result and tears down both subscriptions.

- The notifier is subscribed first. A synchronous notifier next prevents the source from being subscribed at all.
- Notifier completion without next does not stop the source.
- Source completion also completes the result; a source or notifier error terminates it with that error.

```
of(1, 2, 3).pipe(takeUntil(EMPTY)) // next: 1, 2, 3; complete; notifier completion does not stop values
```

Scope. The notifier also accepts other ObservableInput values; its payload is ignored.

Related: skipUntil / take / takeWhile / endWith

rxjs | API docs | Pinned source | ID: takeUntil

takeWhile

PIPEABLE / CURRENT / COURSE LIST

```
takeWhile(predicate: A => boolean, inclusive?: boolean): A* => A*
```

Forward the initial run of values that satisfy the predicate. Complete at the first false result, optionally including that boundary value.

- inclusive defaults to false; true emits the first failing value before completion.
- After the first failure, the source is unsubscribed and no later values are tested.
- If the source completes while all tested values pass, completion is forwarded normally.

```
of(1, 2, 3, 1).pipe(takeWhile(n => n < 3)) // next: 1, 2; complete when 3 arrives
```

Scope. The predicate also receives a zero-based index. Type-guard and Boolean overloads can narrow the result when inclusive is false or omitted; including the failing boundary generally preserves A.

Related: skipWhile / filter / takeUntil / take

rxjs | API docs | Pinned source | ID: takeWhile

throttle

PIPEABLE / CURRENT / COURSE LIST

```
throttle(durationSelector: A => unknown*, config?: ThrottleConfig): A* => A*
```

Use a duration to limit which values pass during a burst. By default, emit a leading value immediately and suppress further values while its duration is active.

- Defaults are leading: true and trailing: false. With trailing enabled, remember the latest suppressed value for release on a duration next.
- A duration next ends the current gate. A trailing emission starts a new duration while the source is still active.
- Duration completion without next ends the gate without a trailing emission.
- Source completion waits only if a trailing value is pending behind an active duration; its next releases that value, while completion without next discards it.

```
of(1, 2, 3).pipe(throttle(() => timer(10))) // next: 1; complete
```

Scope. ThrottleConfig has optional leading and trailing booleans. The duration selector also accepts other ObservableInput values; their payloads are ignored.

Related: throttleTime / audit / debounce

rxjs | API docs | Pinned source | ID: throttle

throttleTime

PIPEABLE / CURRENT / COURSE LIST

```
throttleTime(duration: number, scheduler?: SchedulerLike, config?: ThrottleConfig): A* => A*
```

By default, emit a value immediately and suppress new values for a fixed duration. Leading and trailing options choose which ends of each interval may emit.

- Defaults are leading: true and trailing: false. trailing: true can emit the latest value held during the interval.
- Suppressed source values do not restart the running interval. A trailing emission can start the next interval while the source remains active.
- If source completion leaves a trailing value pending, the result waits for the timer to release it; the default configuration completes immediately.
- The scheduler defaults to asyncScheduler, with time units normally in milliseconds. Errors and downstream unsubscription discard pending values.

```
of(1, 2, 3).pipe(throttleTime(10)) // next: 1; complete
```

Scope. Equivalent to throttle(() => timer(duration, scheduler), config). ThrottleConfig contains optional leading and trailing booleans.

Related: throttle / auditTime / debounceTime / sampleTime

rxjs | API docs | Pinned source | ID: throttleTime

05 / 20 entries

Join Operators

Combine a primary source with companions, or flatten an outer stream. Read who triggers output and when inner subscriptions start.

combineLatestAll

PIPEABLE / CURRENT / COURSE LIST

```
combineLatestAll(): (A*)* => (A[])*
```

Collect the inner sources until the outer source completes, then combine those sources by their latest values.

- No inner source is subscribed before outer completion; a never-completing outer prevents all inner subscriptions.
- After collection, every inner must provide a first value. Later inner values emit a new array in collected-source order.
- A completed inner's latest value remains usable; normal result completion waits for all collected inners to complete.
- An empty outer completes without a next value. Hot inner values emitted before the delayed inner subscriptions are not recovered.

```
of(of(1, 2), of(10, 20)).pipe(combineLatestAll()) // next: [2, 10], [2, 20]; complete
```

Scope. The selected form uses homogeneous Observable<A> inners and produces array values. Native inputs may be other ObservableInput values, and an optional projection can produce B* instead of (A[])*.

Related: combineLatest / combineLatestWith / zipAll / mergeAll

rxjs | API docs | Pinned source | ID: combineLatestAll

concatAll

PIPEABLE / CURRENT / COURSE LIST

```
concatAll(): (A*)* => A*
```

Subscribe to one inner source at a time. Queue later inner sources and start the next only when the current inner completes.

- The first inner can start before the outer completes; output follows inner subscription order.
- Outer completion waits for the active inner and queued inners to finish. A nonterminating inner blocks the queue.
- The queue stores inner sources, not events missed before their subscriptions; hot inner values can be lost while waiting.
- A fast outer can grow the queue without bound. Errors terminate the result and discard queued work.

```
of(of(1, 2).pipe(delay(10)), of(3)).pipe(concatAll()) // next: 1, 2, 3; complete
```

Scope. Equivalent to mergeAll(1). Native inner inputs may also be other ObservableInput values.

Related: concatMap / mergeAll / switchAll / exhaustAll

`rxjs` | API docs | Pinned source | ID: concatAll

exhaustAll

PIPEABLE / CURRENT / COURSE LIST

```
exhaustAll(): (A*)* => A*
```

Subscribe to an inner source only when no inner is active. Ignore new inner sources while the current one is running.

- Ignored inner sources are neither subscribed nor queued for later.
- When the active inner completes, the next outer value can supply a new inner.
- Outer completion waits for the active inner. A nonterminating active inner keeps the result open.

```
of(of(1, 2).pipe(delay(10)), of(3)).pipe(exhaustAll()) // next: 1, 2; complete; of(3) is ignored
```

Scope. Native inner inputs may also be other ObservableInput values. exhaust is the deprecated earlier name.

Related: exhaustMap / exhaust / concatAll / switchAll

`rxjs` | API docs | Pinned source | ID: exhaustAll

mergeAll

PIPEABLE / CURRENT / COURSE LIST

```
mergeAll(concurrent?: number): (A*)* => A*
```

Subscribe to inner sources as they arrive, up to the concurrency limit, and forward values from all active inners.

- concurrent defaults to Infinity. A positive limit queues additional inner sources until an active inner completes.
- Values follow actual arrival order across active inners; earlier outer positions do not reserve earlier output positions.
- Outer completion waits for all active and queued inners to finish.
- The limit bounds active subscriptions, not queue length. Errors cancel active inners and discard queued work.

```
of(of(1, 2).pipe(delay(10)), of(3)).pipe(mergeAll()) // next: 3, 1, 2; complete
```

Scope. Use a positive integer limit or Infinity. Native inner inputs may also be other ObservableInput values; mergeAll(1) has concatAll's policy.

Related: mergeMap / concatAll / switchAll / exhaustAll

`rxjs` | API docs | Pinned source | ID: mergeAll

switchAll

PIPEABLE / CURRENT / COURSE LIST

```
switchAll(): (A*)* => A*
```

Follow only the latest inner source. Each new inner replaces the previous inner subscription immediately.

- The previous inner is unsubscribed before its replacement is subscribed; already emitted values remain part of the output history.
- Outer completion waits for the current inner to complete; it does not cancel that final inner.
- Stopping underlying external work depends on the inner source's teardown implementation.

```
of(of(1, 2).pipe(delay(10)), of(3)).pipe(switchAll()) // next: 3; complete; the delayed inner is unsubscribed
```

Scope. Equivalent to switchMap(inner => inner). Native inner inputs may also be other ObservableInput values.

Related: switchMap / mergeAll / concatAll / exhaustAll

`rxjs` | API docs | Pinned source | ID: switchAll

startWith

PIPEABLE / CURRENT / COURSE LIST

```
startWith(...values: B[]): A* => (A | B)*
```

On subscription, emit the supplied values in order, then subscribe to the source and forward its values.

- In the selected form, the supplied values are emitted synchronously before the source subscription starts.
- If downstream closes while receiving those initial values, the source need not be subscribed.
- Argument expressions are evaluated when constructing the operator. Use defer when a starting value must be computed anew per subscription.

```
of(2, 3).pipe(startWith(0, 1)) // next: 0, 1, 2, 3; complete
```

Scope. The variadic native form preserves unions of heterogeneous initial values, including null and undefined. Only the scheduler-argument overload is deprecated; startWith itself is current.

Related: endWith / concatWith / scan / defer

`rxjs` | API docs | Pinned source | ID: startWith

withLatestFrom

PIPEABLE / CURRENT / COURSE LIST

```
withLatestFrom(other: B*): A* => [A, B]*
```

When the primary source emits A, pair it with the latest B from the companion. Companion values update the remembered B without triggering output.

- Every companion must have emitted once before a primary value can produce a tuple; early primary values are discarded.
- Companions are subscribed before the primary source. A synchronous companion can establish its latest value during setup.
- Companion completion does not complete the result. A completed companion's last value remains available; a companion that completes empty never establishes readiness.
- Primary completion completes the result and unsubscribes companions; an error from any subscribed input terminates it.

```
of(1, 2).pipe(withLatestFrom(of('a', 'b'))) // next: [1, 'b'], [2, 'b']; complete
```

Scope. The selected form has one Observable companion. Native forms accept multiple ObservableInput companions and an optional projection; output tuples keep the primary value first.

Related: combineLatestWith / sample / zipWith

`rxjs` | API docs | Pinned source | ID: withLatestFrom

combineAll

PIPEABLE / DEPRECATED / SUPPLEMENT

```
combineAll(): (A*)* => (A[])*
```

Collect inner sources until outer completion, then emit arrays of their latest values. This is the older name for combineLatestAll.

- Inner subscriptions begin only after the outer completes.
- Each collected inner must emit before any combined array can be emitted; later arrays retain collected-source order.
- Normal completion waits for all collected inner sources; an empty outer completes without emitting.

Use now: combineLatestAll

```
of(of(1, 2), of(10, 20)).pipe(combineAll()) // next: [2, 10], [2, 20]; complete
```

Scope. The function is a direct alias of combineLatestAll, including its ObservableInput and projection overloads.

Related: combineLatestAll / combineLatest

`rxjs` | API docs | Pinned source | ID: combineAll

combineLatest

PIPEABLE / DEPRECATED / SUPPLEMENT / combineLatest-operator

```
combineLatest(other: B*): A* => [A, B]*
```

Combine the primary source with companions so every input update emits their latest-value tuple once all inputs have emitted.

- This is the legacy pipeable import from rxjs/operators; the creation function exported by rxjs is a separate API.
- Completed inputs retain their latest values; normal completion waits for all inputs. An empty-completing input prevents readiness without forcing immediate completion in RxJS 7.8.2.
- The primary source is subscribed before its companions, which affects synchronous examples.

Use now: combineLatestWith

```
import { combineLatest as combineLatestOperator } from 'rxjs/operators';
of(1, 2).pipe(combineLatestOperator(of('a', 'b'))) // next: [2, 'a'], [2, 'b']; complete
```

Scope. Legacy forms accept an array or separate ObservableInput companions and an optional projection. Prefer combineLatestWith followed by map for a projected result.

Related: combineLatestWith / combineLatest / withLatestFrom

`rxjs/operators` | API docs | Pinned source | ID: combineLatest-operator

combineLatestWith

PIPEABLE / CURRENT / SUPPLEMENT

```
combineLatestWith(other: B*): A* => [A, B]*
```

After both inputs have supplied a value, any new value from either input emits a tuple containing their latest values.

- The primary source is subscribed first, followed by companions in argument order; synchronous inputs can finish during setup.
- A completed input's last value remains usable. Normal result completion waits for all inputs to complete.
- In RxJS 7.8.2, an input that completes empty prevents tuples but does not itself force immediate result completion; another input can keep the result open.
- A source error terminates the result and unsubscribes the other inputs.

```
of(1, 2).pipe(combineLatestWith(of('a', 'b'))) // next: [2, 'a'], [2, 'b']; complete
```

Scope. The native variadic form accepts several ObservableInput companions and produces [A, B, C, ...]* in source order.

Related: combineLatest / withLatestFrom / zipWith / combineLatest-operator

`rxjs` | API docs | Pinned source | ID: combineLatestWith

concat

PIPEABLE / DEPRECATED / SUPPLEMENT / concat-operator

```
concat(other: B*): A* => (A | B)*
```

Forward the primary source, then subscribe to each supplied companion in order after the previous source completes.

- This is the legacy pipeable import from rxjs/operators, distinct from the concat creation function.
- A nonterminating input blocks later subscriptions; an error ends the sequence.
- Hot companion values sent before the companion's turn are not retained.

Use now: concatWith

```
import { concat as concatOperator } from 'rxjs/operators';
of(1, 2).pipe(concatOperator(of('a'))) // next: 1, 2, 'a'; complete
```

Scope. Legacy forms accept multiple ObservableInput companions and an optional final scheduler. Use concatWith for the selected unscheduled form.

Related: concatWith / concat / concatAll

`rxjs/operators` | API docs | Pinned source | ID: concat-operator

concatWith

PIPEABLE / CURRENT / SUPPLEMENT

```
concatWith(other: B*): A* => (A | B)*
```

Forward the primary source, then subscribe to the companion after the primary completes and forward the companion.

- With multiple companions, subscriptions proceed in argument order, one at a time.
- A source that never completes blocks later subscriptions; an error ends the sequence without moving on.
- Values from a hot companion before its turn are not buffered by concatWith.

```
of(1, 2).pipe(concatWith(of('a', 'b'))) // next: 1, 2, 'a', 'b'; complete
```

Scope. The native form accepts multiple ObservableInput companions. The output payload is the union of their value types and the primary source's value type.

Related: concat / concatAll / mergeWith / concat-operator

`rxjs` | API docs | Pinned source | ID: concatWith

endWith

PIPEABLE / CURRENT / SUPPLEMENT

```
endWith(...values: B[]): A* => (A | B)*
```

Forward the source, then emit the supplied values in order after normal source completion and complete the result.

- In the selected form, the appended values are emitted synchronously when source completion is received.
- A source error or downstream unsubscription does not emit the appended values.
- A source that never completes never reaches the appended values.

```
of(1, 2).pipe(endWith('done')) // next: 1, 2, 'done'; complete
```

Scope. The native variadic form preserves unions of heterogeneous appended values. The scheduler-argument behavior is deprecated; endWith itself is current.

Related: startWith / concatWith / finalize / takeUntil

`rxjs` | API docs | Pinned source | ID: endWith

merge

PIPEABLE / DEPRECATED / SUPPLEMENT / merge-operator

```
merge(other: B*): A* => (A | B)*
```

Subscribe to the primary and companion sources and interleave their arriving values into one result.

- This is the legacy pipeable import from rxjs/operators, distinct from the merge creation function.
- The selected form starts inputs in source order and permits overlapping subscriptions without a concurrency limit.
- Normal completion waits for all inputs; an input error terminates the result and cancels the others.

Use now: mergeWith for the common unlimited-concurrency form; use merge or mergeAll for an explicit concurrency limit

```
import { merge as mergeOperator } from 'rxjs/operators';
of(1, 2).pipe(delay(10), mergeOperator(of('a'))) // next: 'a', 1, 2; complete
```

Scope. Legacy forms accept multiple ObservableInput companions, an optional concurrency number, and an optional scheduler. Those extra options are not parameters of mergeWith.

Related: mergeWith / merge / mergeAll

`rxjs/operators` | API docs | Pinned source | ID: merge-operator

mergeWith

PIPEABLE / CURRENT / SUPPLEMENT

```
mergeWith(other: B*): A* => (A | B)*
```

Subscribe to the primary and companion sources and forward each input's values as they arrive.

- Subscriptions start in primary-then-companion order, so a synchronous input can emit during setup before later inputs are subscribed.
- Normal completion waits for every input to complete.
- An error from any input terminates the result and unsubscribes the others.

```
of(1, 2).pipe(delay(10), mergeWith(of('a'))) // next: 'a', 1, 2; complete
```

Scope. The native form accepts multiple ObservableInput companions. For an explicit concurrency limit, use an appropriate merge creation form or a higher-order source with mergeAll(limit).

Related: merge / mergeAll / concatWith / merge-operator

`rxjs` | API docs | Pinned source | ID: mergeWith

race

PIPEABLE / DEPRECATED / SUPPLEMENT / race-operator

```
race(other: B*): A* => (A | B)*
```

Race the primary and companion sources: the first next chooses a source to follow, while a first complete or error ends the result immediately.

- This is the legacy pipeable import from rxjs/operators, distinct from the race creation function.
- Losing subscriptions are closed; a synchronous winner can prevent later inputs from being subscribed.
- An empty input can end the race before any value is emitted.

Use now: raceWith

```
import { race as raceOperator } from 'rxjs/operators';
of(1, 2).pipe(raceOperator(of('later'))) // next: 1, 2; complete
```

Scope. Legacy forms accept an array or separate ObservableInput companions. The implementation delegates to raceWith.

Related: raceWith / race / mergeWith

`rxjs/operators` | API docs | Pinned source | ID: race-operator

raceWith

PIPEABLE / CURRENT / SUPPLEMENT

```
raceWith(other: B*): A* => (A | B)*
```

Let the first input notification decide the result. A first next selects that source for the remaining output; a first complete or error ends the result immediately.

- Once a source wins with a next value, other subscribed inputs are unsubscribed.
- Inputs are subscribed in primary-then-companion order. A synchronous winner can prevent later inputs from being subscribed at all.
- An empty source can win by completing before any competing value arrives.

```
of(1, 2).pipe(raceWith(of('later'))) // next: 1, 2; complete; the companion is never subscribed
```

Scope. The native form accepts multiple ObservableInput companions. With no companions, raceWith() returns the primary source unchanged.

Related: race / mergeWith / race-operator

`rxjs` | API docs | Pinned source | ID: raceWith

zip

PIPEABLE / DEPRECATED / SUPPLEMENT / zip-operator

```
zip(other: B*): A* => [A, B]*
```

Combine equally numbered values from the primary and companion sources into tuples, waiting until every input has its next unpaired value.

- This is the legacy pipeable import from rxjs/operators, distinct from the zip creation function.
- Faster inputs are queued; completion occurs when a completed input has no queued values left.
- Queue growth depends on differences in input rates, so a slow companion can retain many primary values.

Use now: zipWith

```
import { zip as zipOperator } from 'rxjs/operators';
of(1, 2, 3).pipe(zipOperator(of('a', 'b'))) // next: [1, 'a'], [2, 'b']; complete
```

Scope. Legacy forms accept multiple ObservableInput companions and optional projection overloads. Prefer zipWith followed by map for a projected result.

Related: zipWith / zip / zipAll

`rxjs/operators` | API docs | Pinned source | ID: zip-operator

zipAll

PIPEABLE / CURRENT / SUPPLEMENT

```
zipAll(): (A*)* => (A[])*
```

Collect inner sources until the outer completes, then combine their first values, second values, and later matching positions into arrays.

- No inner is subscribed before outer completion; a nonterminating outer prevents the join from starting.
- Array positions follow the order in which inner sources were collected, while each inner's unpaired values wait in a queue.
- The result completes when any completed inner has no queued value left. An empty outer completes without a next value.
- Hot inner values sent before the eventual subscriptions are missed, and faster subscribed inners can grow their queues.

```
of(of(1, 2), of(10, 20, 30)).pipe(zipAll()) // next: [1, 10], [2, 20]; complete
```

Scope. The selected form uses homogeneous Observable<A> inners. Native inputs may be other ObservableInput values; an optional projection changes the output from (A[])* to B*.

Related: zip / zipWith / combineLatestAll

`rxjs` | API docs | Pinned source | ID: zipAll

zipWith

PIPEABLE / CURRENT / SUPPLEMENT

```
zipWith(other: B*): A* => [A, B]*
```

Pair the first value from each input, then the second from each, and so on. Emit a tuple whenever every input has its next unpaired value available.

- Values from faster inputs wait in queues; these queues can grow when arrival rates differ.
- A completed input can still contribute values already queued.
- The result completes as soon as a completed input has no queued value left, because another full tuple is impossible. An input that completes empty can therefore end the result immediately.

```
of(1, 2, 3).pipe(zipWith(of('a', 'b'))) // next: [1, 'a'], [2, 'b']; complete
```

Scope. The native variadic form accepts multiple ObservableInput companions and emits tuples in primary-then-companion order.

Related: zip / zipAll / combineLatestWith / zip-operator

rxjs | API docs | Pinned source | ID: zipWith

06 / 10 entries

Multicasting Operators

Choose how listeners share source connections and what later listeners receive. Type shape alone does not specify this policy.

multicast

CONNECTABLE / DEPRECATED / COURSE LIST

```
multicast(connector: () => Subject<A>): A* => ConnectableObservable<A>
```

Routes one connected source execution through a Subject chosen by the caller. The returned legacy connectable exposes explicit connection control.

- Subscribing to the result attaches a listener; connect() starts the source connection. Unsubscribing the connection tears that connection down.
- A Subject instance reuses that instance; a factory can supply a new one when the legacy connectable recreates its subject. The Subject type controls replay and terminal behavior.

Use now: connectable(source, { connector }) for explicit connections; connect(selector, { connector }) for selector-local sharing.

```
const connected$ = of(1, 2).pipe(multicast(() => new Subject<number>()));
connected$.subscribe(console.log);
connected$.connect(); // next: 1, 2; complete
```

Scope. The selector overload, multicast(connector, shared => result), returns B* and shares within that selector instead of exposing connect().

Related: connectable / connect / share

rxjs | API docs | Pinned source | ID: multicast

publish

CONNECTABLE / DEPRECATED / COURSE LIST

```
publish(): A* => ConnectableObservable<A>
```

Creates a legacy connectable backed by one plain Subject, sharing live values without replay.

- Listeners can attach before connect() starts the source. The plain Subject has no cached next value for a late listener.
- The no-selector form reuses its Subject. Its completed or errored Subject is retained for later listeners.

Use now: connectable(source, { connector: () => new Subject(), resetOnDisconnect: false }) for the legacy connection policy; share() for default automatic sharing.

```
const connected$ = of(1, 2).pipe(publish());
connected$.subscribe(console.log);
connected$.connect(); // next: 1, 2; complete
```

Scope. publish(selector: A* => B*) returns A* => B*; use connect(selector) for that form.

Related: multicast / connectable / connect / share

rxjs | API docs | Pinned source | ID: publish

publishBehavior

CONNECTABLE / DEPRECATED / COURSE LIST

```
publishBehavior(seed: A): A* => ConnectableObservable<A>
```

Creates a legacy connectable backed by a BehaviorSubject. While the subject is active, a listener immediately receives its current value, initially the seed.

- The seed can be observed before connect() starts the source; source values subsequently replace the current value.
- A late listener after completion receives only complete, not the cached value. After error it receives that error.

Use now: connectable(source, { connector: () => new BehaviorSubject(seed), resetOnDisconnect: false }); choose share configuration explicitly for automatic connections.

```
const connected$ = of(1, 2).pipe(publishBehavior(0));
connected$.subscribe(console.log); // next: 0
connected$.connect(); // next: 1, 2; complete
```

Related: publishReplay / connectable / share

rxjs | API docs | Pinned source | ID: publishBehavior

publishLast

CONNECTABLE / DEPRECATED / COURSE LIST

```
publishLast(): A* => ConnectableObservable<A>
```

Creates a legacy connectable backed by an AsyncSubject. It remembers the latest source value and publishes that value only when the source completes normally.

- An empty source completes without a next value; a source error is forwarded without publishing the remembered value.
- After successful completion, late listeners receive the final value and complete. Before completion, source values are withheld.

Use now: connectable(source, { connector: () => new AsyncSubject(), resetOnDisconnect: false }).

```
const connected$ = of(1, 2, 3).pipe(publishLast());
connected$.subscribe(console.log);
connected$.connect(); // next: 3; complete
```

Related: last / connectable / share

rxjs | API docs | Pinned source | ID: publishLast

publishReplay

CONNECTABLE / DEPRECATED / COURSE LIST

```
publishReplay(bufferSize?: number, windowTime?: number): A* => ConnectableObservable<A>
```

Creates a legacy connectable backed by one ReplaySubject. New listeners first receive eligible cached values, then live values.

- The buffer is limited by count and age; both defaults are unbounded. connect() starts the source; replay itself does not start it.
- The same ReplaySubject is retained, including its completion or error. Cached values within the retention limits can be replayed before that terminal notification.

Use now: connectable(source, { connector: () => new ReplaySubject(bufferSize, windowTime), resetOnDisconnect: false }); shareReplay has different reset defaults.

```
const connected$ = of(1, 2, 3).pipe(publishReplay(2));
const connection = (connected$ as ConnectableObservable<number>).connect();
connected$.subscribe(console.log); // next: 2, 3; complete
```

Scope. This card shows runtime connectability. The no-selector public overload is typed only as MonoTypeOperatorFunction<A>, so TypeScript can require the cast shown. Selector and timestamp-provider overloads also exist.

Related: shareReplay / publishBehavior / connectable

rxjs | API docs | Pinned source | ID: publishReplay

share

PIPEABLE / CURRENT / COURSE LIST

```
share(config?: ShareConfig<A>): A* => A*
```

Shares a source connection among listeners attached to the same shared Observable instance. The default connector is a plain Subject, so it forwards live values without replay.

- By default the first listener connects, and losing the last listener disconnects an active source. Error, completion, and a zero listener count each reset the sharing state.
- Separate subscriptions to a synchronously completing shared source can each start a new execution because the first execution has already completed and reset.
- The configuration changes the connector and reset policies; it can retain a terminal state or delay a reset.

```
const shared$ = interval(1000).pipe(share());
// Overlapping subscriptions to shared$ use one interval connection.
```

Scope. The signature preserves payload type; sharing is a connection policy, not a property of A* itself.

Related: shareReplay / connect / refCount

rxjs | API docs | Pinned source | ID: share

connect

PIPEABLE / CURRENT / SUPPLEMENT

```
connect(selector: A* => B*, config?: ConnectConfig<A>): A* => B*
```

Gives a selector a shared view of the source, then connects the source after the selected result has been subscribed. This lets several branches observe one source execution within that result subscription.

- A new connector is made for each subscription to the resulting Observable. This does not automatically share across separate result subscriptions.
- The default connector is a plain Subject. The selected result supplies the output type and termination policy; tearing down the result also tears down its source connection.

```
of(1, 2).pipe(connect(shared$ =>
  merge(shared$.pipe(map(n => n * 10)), shared$.pipe(map(n => n + 1)))
)) // next: 10, 2, 20, 3; complete
```

Scope. The actual selector may return any ObservableInput<B>. ConnectConfig<A> requires a connector when a config object is supplied.

Related: share / connectable / multicast

rxjs | API docs | Pinned source | ID: connect

connectable

CONNECTABLE / CURRENT / SUPPLEMENT

```
connectable(source: A*, config?: ConnectableConfig<A>): Connectable<A>
```

Create a shared stream with an explicit connection switch. Subscribers attach to the shared subject; connect() starts the source subscription.

- The default connector creates a plain Subject, so it does not replay earlier values to later subscribers.
- connect() returns the shared connection Subscription. Unsubscribing that connection disconnects the source; merely having no observers does not automatically disconnect it.
- By default, resetOnDisconnect is true: ending a connection replaces the subject for future subscriptions and connections. Existing observers are not moved to the replacement subject.
- Attach observers before connecting when synchronous source values must reach them.

```
const shared = connectable(of(1, 2));
shared.subscribe(console.log); // no source values yet
shared.connect(); // next: 1, 2; complete
```

Scope. The actual source may be ObservableInput. Connectable<A> is an A* with connect(): Subscription. A supplied config provides a connector factory and may select resetOnDisconnect; replay requires an appropriate connector.

Related: share / connect / multicast

rxjs | API docs | Pinned source | ID: connectable

refCount

PIPEABLE / DEPRECATED / SUPPLEMENT

```
refCount(): ConnectableObservable<A> => A*
```

Adds listener-count control to a legacy ConnectableObservable: connect for the first listener and disconnect an active shared connection after the last listener leaves.

- The connector and its retained terminal state come from the preceding legacy multicasting operation; refCount does not add replay or choose reset behavior.
- This teaching input names the legacy connectable requirement even though the public TypeScript return type is MonoTypeOperatorFunction<A>. Do not treat it as a generic adapter for the newer connectable() result.

Use now: share(), with an explicit connector/reset configuration when preserving legacy behavior.

```
const shared$ = interval(1000).pipe(publish(), refCount());
// Legacy automatic connection pattern; prefer share().
```

Related: share / publish / connectable

rxjs | API docs | Pinned source | ID: refCount

shareReplay

PIPEABLE / CURRENT / SUPPLEMENT

```
shareReplay(config: ShareReplayConfig): A* => A*
```

Shares through a ReplaySubject so late listeners receive eligible cached values before live values.

- Prefer an explicit bufferSize and refCount. With refCount: true, losing the last listener resets and disconnects an active source; refCount: false keeps that active connection.
- In RxJS 7.8.2, an error resets the shared state, but successful completion retains the replay cache. refCount: true does not clear a successfully completed cache.
- windowTime limits the age of replayed values; expiration is checked by the ReplaySubject and does not by itself trigger a new source execution.

```
const shared$ = of(1, 2, 3).pipe(shareReplay({ bufferSize: 1, refCount: true }));
shared$.subscribe(console.log); // next: 1, 2, 3; complete
shared$.subscribe(console.log); // next: 3; complete
```

Scope. The positional overload shareReplay(bufferSize?, windowTime?, scheduler?) defaults refCount to false. The scheduler supplies the replay timestamp clock; it does not schedule delivery of cached values.

Related: share / publishReplay

rxjs | API docs | Pinned source | ID: shareReplay

07 / 5 entries

Error Handling Operators

Choose whether an error terminates, selects another input, or requests another subscription.

catchError

PIPEABLE / CURRENT / COURSE LIST

```
catchError(recover: unknown => B*): A* => (A | B)*
```

Forwards source values until an error, then replaces the failed subscription with the Observable returned by recover.

- Values emitted before the error remain in the output. Returning a fallback does not undo those values or resume the failed source.
- A thrown recovery error, or an error from the replacement, propagates downstream. Unsubscription tears down whichever subscription is active.

```
concat(of(1), throwError(() => new Error('failed'))).pipe(
  catchError(() => of('fallback'))
) // next: 1, 'fallback'; complete
```

Scope. The actual callback receives (error, caught) and returns ObservableInput<B>. Returning caught requests resubscription through the same catchError logic.

Related: retry / onErrorResumeNextWith

rxjs | API docs | Pinned source | ID: catchError

retry

PIPEABLE / CURRENT / COURSE LIST

```
retry(config?: number | RetryConfig): A* => A*
```

On source error, subscribes to the source again, up to the retry policy. Every attempt can contribute values to the same output.

- retry(2) allows the original subscription plus two retries. The default count is unbounded; count <= 0 does not retry. Completion is forwarded normally.
- A delay may be a duration or an error/count callback. The first delay-notifier next retries; completion without a next completes the result, and a notifier error errors it.
- resetOnSuccess resets the retry counter on a next value, not on completion; it defaults to false. Resubscription repeats source side effects.

```
defer(() => of(1)).pipe(retry(2)) // next: 1; complete; no retry needed
```

Scope. RetryConfig contains count?, delay?, and resetOnSuccess?. The callback retry count starts at 1.

Related: repeat / retryWhen / catchError

rxjs | API docs | Pinned source | ID: retry

retryWhen

PIPEABLE / DEPRECATED / COURSE LIST

```
retryWhen(notifier: unknown* => unknown*): A* => A*
```

Feeds source errors into a notifier factory. Notifier next values request new subscriptions to the source.

- The notifier is created after the first source error. Its output controls retries; its own next payload is ignored.
- Notifier completion completes the result and notifier error errors it. Ordinary source completion completes the result without requiring notifier completion.

Use now: retry({ delay: (error, retryCount) => retrySignal$ }) for the usual retry policy; translate custom notifier state deliberately.

```
request$.pipe(retryWhen(errors$ => errors$.pipe(delay(1000))))
// After each source error, the notifier requests a retry after 1 second.
```

Scope. The notifier may return any ObservableInput. Errors are typed unknown in this teaching view instead of the public any.

Related: retry

rxjs | API docs | Pinned source | ID: retryWhen

onErrorResumeNext

PIPEABLE / DEPRECATED / SUPPLEMENT / onErrorResumeNext-operator

```
onErrorResumeNext(next: B*): A* => (A | B)*
```

The legacy pipeable name for onErrorResumeNextWith. It advances to the next input when the current input completes or errors.

- This entry is the rxjs/operators export. The same name imported from rxjs is a creation function that takes all sources directly.
- Errors from constituent inputs are swallowed; a nonterminating input prevents progression.

Use now: onErrorResumeNextWith from rxjs.

```
// import { onErrorResumeNext } from 'rxjs/operators';
throwError(() => new Error('failed')).pipe(onErrorResumeNext(of(2))) // next: 2; complete
```

Related: onErrorResumeNextWith / onErrorResumeNext

`rxjs/operators` | API docs | Pinned source | ID: onErrorResumeNext-operator

onErrorResumeNextWith

PIPEABLE / CURRENT / SUPPLEMENT

```
onErrorResumeNextWith(next: B*): A* => (A | B)*
```

Concatenates subscriptions, advancing to the next input after either normal completion or error. Errors from the constituent inputs are swallowed.

- Inputs are subscribed sequentially. An input that never terminates prevents reaching later inputs.
- There is no recovery callback and no error value to inspect here; use catchError when the error should determine the fallback.

```
throwError(() => new Error('failed')).pipe(onErrorResumeNextWith(of('next')))
// next: 'next'; complete
```

Scope. Variadic and array input forms accept ObservableInput values and produce the union of input payloads.

Related: onErrorResumeNext / catchError / concatWith

`rxjs` | API docs | Pinned source | ID: onErrorResumeNextWith

08 / 15 entries

Utility Operators

Observe execution, schedule work or delivery, measure time, enforce deadlines, or convert notifications.

tap

PIPEABLE / CURRENT / COURSE LIST

```
tap(effect: A => void): A* => A*
```

Runs an effect for each source value and forwards that same value. The callback return value is ignored.

- A synchronous exception in the next-value callback becomes an error downstream. Teardown hooks have teardown-error behavior instead. Callbacks can still mutate referenced objects.
- tap does not start a subscription by itself. Its observer-object form also provides error, complete, subscribe, unsubscribe, and finalize hooks.

```
of(1, 2).pipe(tap(n => console.log(n))) // next: 1, 2; complete; also logs each value
```

Scope. Use an observer object for multiple hooks. The older separate next/error/complete-argument overload is deprecated; tap itself is current.

Related: map / finalize

`rxjs` | API docs | Pinned source | ID: tap

delay

PIPEABLE / CURRENT / COURSE LIST

```
delay(due: number | Date, scheduler?: SchedulerLike): A* => A*
```

Holds source values until their scheduled release time. A numeric due delays each value by that duration; a Date is an absolute release time.

- The default scheduler is asyncScheduler. The source is subscribed immediately; this delays delivery, not the start of source work.
- Completion waits for held values. A source error is forwarded immediately and discards held values; unsubscription also discards them.
- An empty source can complete immediately: delay does not simply shift every terminal notification by the due time.

```
of(1, 2).pipe(delay(100)) // after about 100 ms: next 1, 2; complete
```

Related: delayWhen / observeOn / subscribeOn

rxjs | API docs | Pinned source | ID: delay

delayWhen

PIPEABLE / CURRENT / COURSE LIST

```
delayWhen(release: A => unknown*): A* => A*
```

Creates a release Observable for each source value. The first next from that Observable releases that value.

- Each value has its own subscription, so later source values can be released first. Output order need not match source order.
- A release Observable that completes without a next drops its value in RxJS 7.8.2. Source completion waits for outstanding releases; errors terminate and discard pending values.

```
of(1, 2).pipe(delayWhen(n => timer(n === 1 ? 20 : 10)))
// next: 2 at about 10 ms, 1 at about 20 ms; complete
```

Scope. The actual callback also receives an index and may return ObservableInput. The deprecated subscriptionDelay overload gates the initial source subscription on notifier next or completion.

Related: delay / debounce

rxjs | API docs | Pinned source | ID: delayWhen

dematerialize

PIPEABLE / CURRENT / COURSE LIST

```
dematerialize(): Notice<A>* => A*
```

Interprets notification objects as real next, error, and complete notifications.

- An N notice emits its value. An E notice errors the result; a C notice completes it. Either terminal notice ends the subscription and prevents later notices from being delivered.
- The input stream can also terminate through its own error or completion channel; a terminal notice is not required.

```
of(1, 2).pipe(materialize(), dematerialize()) // next: 1, 2; complete
```

Scope. The actual input is ObservableNotification<A>, including structurally compatible objects. The output type extracts the value of N notices.

Related: materialize

rxjs | API docs | Pinned source | ID: dematerialize

materialize

PIPEABLE / CURRENT / COURSE LIST

```
materialize(): A* => Notice<A>*
```

Turns next, error, and complete notifications into ordinary next values describing those notifications.

- A source error becomes one E notice followed by normal completion of the materialized result. Normal source completion becomes one C notice followed by completion.
- Unsubscription creates no notice. Materializing an outer stream does not materialize notifications inside its inner streams.

```
of(1).pipe(materialize()) // next: N(1), C; complete
// N(1) and C describe notification objects here, not function calls.
```

Scope. Notice<A> denotes the observable-notification union. Runtime results are Notification<A> instances that also satisfy ObservableNotification<A>.

Related: dematerialize

rxjs | API docs | Pinned source | ID: materialize

observeOn

PIPEABLE / CURRENT / COURSE LIST

```
observeOn(scheduler: SchedulerLike, delay?: number): A* => A*
```

Schedules delivery of next, error, and complete notifications to the downstream listener.

- It does not schedule when source subscription begins or move source work to another thread. The chosen scheduler determines execution timing.
- Delay defaults to 0. Unsubscription cancels pending scheduled deliveries. Unlike delay, an error is scheduled too.

```
of(1, 2).pipe(observeOn(asyncScheduler))
// Schedules next 1, next 2, and complete after the current synchronous work.
```

Related: subscribeOn / delay

rxjs | API docs | Pinned source | ID: observeOn

subscribeOn

PIPEABLE / CURRENT / COURSE LIST

```
subscribeOn(scheduler: SchedulerLike, delay?: number): A* => A*
```

Schedules the act of subscribing to the source, including the work performed as that subscription starts.

- Once the source starts, its own behavior determines how values are delivered. This is not a per-notification scheduling policy.
- Delay defaults to 0. Unsubscribing before the scheduled subscription runs cancels it, so source work does not start.

```
defer(() => of(1, 2)).pipe(subscribeOn(asyncScheduler))
// The factory is called when the scheduled source subscription runs.
```

Related: observeOn / defer

rxjs | API docs | Pinned source | ID: subscribeOn

timeInterval

PIPEABLE / CURRENT / COURSE LIST

```
timeInterval(scheduler?: SchedulerLike): A* => { value: A; interval: number }*
```

Pairs each source value with the elapsed clock time since the preceding source value, or since subscription for the first value.

- The default clock is asyncScheduler.now(). interval is a duration, not an absolute timestamp.
- The operator measures time without delaying values or scheduling source work. Clock units follow the supplied scheduler.

```
interval(100).pipe(take(2), timeInterval())
// Values 0 and 1 are wrapped with intervals of about 100 ms each.
```

Scope. The actual result is a TimeInterval<A> object with these public fields.

Related: timestamp

rxjs | API docs | Pinned source | ID: timeInterval

timestamp

PIPEABLE / CURRENT / COURSE LIST

```
timestamp(clock?: TimestampProvider): A* => { value: A; timestamp: number }*
```

Pairs each source value with the current clock reading at the point this operator receives it.

- By default the clock uses Date.now(). timestamp is an absolute clock reading, unlike the elapsed duration from timeInterval.
- A custom TimestampProvider only needs now(). Reading a clock neither delays delivery nor changes source scheduling.

```
of("ready").pipe(timestamp())
// next: { value: "ready", timestamp: currentClockReading }; complete
```

Scope. The actual result type is Timestamp<A>. A SchedulerLike can serve as a TimestampProvider.

Related: timeInterval / observeOn

rxjs | API docs | Pinned source | ID: timestamp

timeout

PIPEABLE / CURRENT / COURSE LIST

```
timeout(due: number | Date, scheduler?: SchedulerLike): A* => A*
timeout(config: TimeoutConfig<A, B*> & { with: TimeoutInfo<A> => B* }): A* => (A | B)*
```

Enforces a deadline for receiving a source value. On expiry it unsubscribes the source and either errors or switches to a configured fallback.

- For a positive numeric due, the deadline covers the first value and every following gap. A Date overload sets only the first-value deadline. asyncScheduler is the default scheduler.
- The config separates first and each. Without with, expiry raises TimeoutError; with supplies the replacement subscription. Values already forwarded remain in the result.
- The fallback is not recursively checked by the same timeout. Completion or error before a deadline ends the source normally for that channel.

```
NEVER.pipe(timeout({ first: 100, with: () => of("fallback") }))
// next: "fallback" at about 100 ms; complete
```

Scope. At least first or each is required at runtime. The full API accepts ObservableInput fallbacks and metadata. The zero/negative each edge is not represented by the positive-duration teaching rule.

Related: timeoutWith / takeUntil / catchError

rxjs | API docs | Pinned source | ID: timeout

timeoutWith

PIPEABLE / DEPRECATED / COURSE LIST

```
timeoutWith(due: number | Date, fallback: B*, scheduler?: SchedulerLike): A* => (A | B)*
```

The older timeout form that switches to a supplied fallback when a source-value deadline expires.

- A positive number guards the first value and subsequent gaps; a Date guards the first value only.
- On expiry the source is unsubscribed and fallback is subscribed. Previously delivered values remain in the output.

Use now: timeout({ each: duration, with: () => fallback }) or timeout({ first: date, with: () => fallback }).

```
NEVER.pipe(timeoutWith(100, of("fallback")))
// next: "fallback" at about 100 ms; complete
```

Scope. The actual fallback may be any ObservableInput<B>.

Related: timeout

rxjs | API docs | Pinned source | ID: timeoutWith

toArray

PIPEABLE / CURRENT / COURSE LIST

```
toArray(): A* => (A[])*
```

Collects source values into one array and emits that array when the source completes normally.

- An empty completed source emits [] once. Source errors and unsubscription discard the accumulated array.
- A source that never completes never emits its array. Memory use grows with the number of collected values.

```
of(1, 2, 3).pipe(toArray()) // next: [1, 2, 3]; complete
```

Scope. The result is a stream carrying an array value; it is not an array of Observables.

Related: buffer / reduce

rxjs | API docs | Pinned source | ID: toArray

finalize

PIPEABLE / CURRENT / SUPPLEMENT

```
finalize(cleanup: () => void): A* => A*
```

Registers cleanup for the subscription. It runs when that subscription ends through completion, error, or unsubscription.

- The callback receives no reason or last value. It is not a complete callback: cancellation also runs it.
- Placement around sharing matters: upstream cleanup follows the shared source connection; downstream cleanup follows each listener subscription.

```
NEVER.pipe(finalize(() => console.log("released"))).subscribe().unsubscribe();
// No next or complete; logs "released" during teardown.
```

Related: tap / share

rxjs | API docs | Pinned source | ID: finalize

repeat

PIPEABLE / CURRENT / SUPPLEMENT

```
repeat(config?: number | RepeatConfig): A* => A*
```

On normal source completion, subscribes again according to the repeat policy. Source errors still terminate the result.

- repeat(3) means three total source subscriptions, including the first. The default is unbounded; count <= 0 completes without subscribing to the source.
- A delay may be a duration or a completed-run-count callback. Its first next starts the next run; completion without a next ends repetition.
- Resubscription repeats source side effects. Unbounded repetition of a synchronous source needs particular care; it need not yield execution.

```
of(1, 2).pipe(repeat(2)) // next: 1, 2, 1, 2; complete
```

Scope. RepeatConfig contains count? and delay?. The delay callback first receives 1 after the first run completes.

Related: retry / repeatWhen

rxjs | API docs | Pinned source | ID: repeat

repeatWhen

PIPEABLE / DEPRECATED / SUPPLEMENT

```
repeatWhen(notifier: void* => unknown*): A* => A*
```

Feeds source completion signals into a notifier factory. Each notifier next requests another source subscription.

- Source errors are forwarded. Notifier errors also error the result.
- Notifier completion stops requesting new runs; the result completes once the active source run has also completed. This differs from retryWhen notifier completion.
- With synchronous sources and synchronous notifier completion, notification order can prevent a requested repeat from starting. Test that boundary when translating legacy notifier logic.

Use now: repeat({ delay: completedRunCount => repeatSignal$ }) for the usual repetition policy.

```
timer(10).pipe(repeatWhen(completions$ => completions$.pipe(take(1))))
// next: 0, then 0 from the repeated run; complete
```

Scope. The void values are next notifications signalling completed runs, not the completion channel itself. The actual notifier return type is ObservableInput.

Related: repeat / retryWhen

rxjs | API docs | Pinned source | ID: repeatWhen

09 / 7 entries

Conditional and Boolean Operators

Answer questions about a subscription. Many answers require completion; a counterexample can sometimes end the search early.

defaultIfEmpty

PIPEABLE / CURRENT / COURSE LIST

```
defaultIfEmpty(fallback: B): A* => (A | B)*
```

Forwards source values. If the source completes normally without any value, it emits the fallback before completing.

- null and undefined are ordinary values and therefore prevent the fallback from being emitted.
- It does not recover errors and does not impose a timeout on a silent, open source.

```
EMPTY.pipe(defaultIfEmpty("none")) // next: "none"; complete
```

Related: throwIfEmpty / startWith

rxjs | API docs | Pinned source | ID: defaultIfEmpty

every

PIPEABLE / CURRENT / COURSE LIST

```
every(predicate: A => boolean): A* => boolean*
```

Emits false and completes at the first value that fails the predicate. If all received values pass and the source completes, it emits true.

- An empty completed source emits true. An open source whose values all pass has not yet established true.
- The first failing value ends the subscription early. Predicate exceptions error the result.

```
of(2, 4, 5, 6).pipe(every(n => n % 2 === 0)) // next: false; complete at 5
```

Scope. The actual predicate also receives index and source. Boolean-constructor overloads are omitted.

Related: find / isEmpty

rxjs | API docs | Pinned source | ID: every

find

PIPEABLE / CURRENT / COURSE LIST

```
find(predicate: A => boolean): A* => (A | undefined)*
```

Emits the first matching value and completes. If the source completes without a match, it emits undefined and completes.

- A successful match unsubscribes early. A nonterminating source with no match can remain open indefinitely.
- undefined is a delivered payload here, not an empty stream. It is indistinguishable by value from a matched undefined unless the domain supplies a tag.

```
of(1, 3).pipe(find(n => n % 2 === 0)) // next: undefined; complete
```

Scope. Type-guard predicates narrow the successful payload to B; the result still includes undefined. Index and source callback parameters are omitted.

Related: first / findIndex / filter

rxjs | API docs | Pinned source | ID: find

findIndex

PIPEABLE / CURRENT / COURSE LIST

```
findIndex(predicate: A => boolean): A* => number*
```

Emits the zero-based index of the first matching source value and completes. If the source completes with no match, it emits -1.

- The index counts all received source values, including those that do not match.
- A match ends the source subscription early; without a match, normal completion is needed to conclude -1.

```
of("a", "b", "c").pipe(findIndex(x => x === "b")) // next: 1; complete
```

Scope. The actual predicate also receives index and source.

Related: find

rxjs | API docs | Pinned source | ID: findIndex

isEmpty

PIPEABLE / CURRENT / COURSE LIST

```
isEmpty(): A* => boolean*
```

Emits false and completes as soon as a source value arrives. Emits true only when the source completes without a value.

- A source error before a value is an error, not true. A silent source that never completes leaves the result open.
- The false outcome needs only the first value and cancels further source delivery.

```
of(undefined).pipe(isEmpty()) // next: false; complete
```

Related: every / defaultIfEmpty

rxjs | API docs | Pinned source | ID: isEmpty

sequenceEqual

PIPEABLE / CURRENT / SUPPLEMENT

```
sequenceEqual(other: A*, equal?: (A, A) => boolean): A* => boolean*
```

Compares the two sequences by value position, ignoring differences in their emission times. It emits one boolean once equality or inequality can be decided.

- Both streams must complete with matching lengths and values to establish true. A mismatching pair or an extra value after the other completes can establish false early.
- The default equality is ===. Faster input values are buffered while awaiting partners, so a large rate imbalance can grow memory.

```
of(1, 2).pipe(sequenceEqual(of(1, 2))) // next: true; complete
```

Scope. The actual comparison input may be ObservableInput<A>.

Related: zip / every

rxjs | API docs | Pinned source | ID: sequenceEqual

throwIfEmpty

PIPEABLE / CURRENT / SUPPLEMENT

```
throwIfEmpty(errorFactory?: () => unknown): A* => A*
```

Passes source values through, but replaces an empty normal completion with an error.

- The default factory creates EmptyError. It runs only if completion arrives before any next value.
- This does not impose a deadline: a source that stays open without values keeps the result open.

```
EMPTY.pipe(throwIfEmpty()) // no next; error: EmptyError
```

Related: defaultIfEmpty / first

rxjs | API docs | Pinned source | ID: throwIfEmpty

10 / 4 entries

Mathematical and Aggregate Operators

Produce a final aggregate when the source completes. Compare with scan when learners need every intermediate state.

count

PIPEABLE / CURRENT / COURSE LIST

```
count(predicate?: A => boolean): A* => number*
```

Counts received source values, or only the values satisfying a predicate, and emits the total on normal completion.

- An empty completed source emits 0. No running counts are emitted.
- A source error prevents a final count. Use scan when the lesson requires a running count.

```
of(1, 2, 3, 4).pipe(count(n => n % 2 === 0)) // next: 2; complete
```

Scope. The optional predicate also receives the zero-based source index.

Related: scan / reduce

rxjs | API docs | Pinned source | ID: count

max

PIPEABLE / CURRENT / COURSE LIST

```
max(compare?: (A, A) => number): A* => A*
```

Remembers the greatest source value and emits it on normal completion.

- An empty completed source emits no value. No running maximum is emitted before completion.
- Without a comparer it uses JavaScript >. A comparer returns a positive number when its first argument should rank higher; ties select the later value in RxJS 7.8.2.

```
of(3, 1, 5, 2).pipe(max()) // next: 5; complete
```

Related: min / reduce / scan

rxjs | API docs | Pinned source | ID: max

min

PIPEABLE / CURRENT / COURSE LIST

```
min(compare?: (A, A) => number): A* => A*
```

Remembers the least source value and emits it on normal completion.

- An empty completed source emits no value. No running minimum is emitted before completion.
- Without a comparer it uses JavaScript <. A comparer returns a negative number when its first argument should rank lower; ties select the later value in RxJS 7.8.2.

```
of(3, 1, 5, 2).pipe(min()) // next: 1; complete
```

Related: max / reduce / scan

rxjs | API docs | Pinned source | ID: min

reduce

PIPEABLE / CURRENT / COURSE LIST

```
reduce(reducer: (S, A) => S, seed: S): A* => S*
```

Updates accumulated state for each source value, then emits only the final state on normal completion.

- With the seed shown here, an empty completed source emits the seed once. scan with the same seed emits nothing for that empty source.
- The seed is an initial value, not a deep clone; use an immutable reducer. Error or unsubscription prevents the final state from being emitted.

```
of(1, 2, 3).pipe(reduce((sum, n) => sum + n, 0)) // next: 6; complete
```

Scope. Without a seed, the first input initializes state without calling the reducer. In RxJS 7.8.2 an empty source with that overload completes without a value. The reducer also receives an index.

Related: scan / toArray / count

rxjs | API docs | Pinned source | ID: reduce

Named types and supporting notation

Notation	Read it as
`SchedulerLike`	A scheduler supplies now() and scheduling operations. It can represent real or virtual time; choosing one does not imply another thread.
`TimestampProvider`	A clock object with now(): number. It measures time and need not schedule work.
`ThrottleConfig`	An object with optional leading and trailing booleans. Defaults: leading true, trailing false.
`RetryConfig / RepeatConfig`	Retry uses count?, delay?, resetOnSuccess?; repeat uses count? and delay?. Retry count means retries after the original run; repeat count means total runs.
`ShareConfig<A>`	Selects a connector and resetOnError, resetOnComplete, and resetOnRefCountZero policies. Each reset can be a boolean or a callback returning a reset notifier.
`ShareReplayConfig`	Requires refCount in the typed object form; bufferSize, windowTime, and scheduler are optional. Prefer explicit bounded buffering and an explicit refCount decision.
`ConnectConfig<A>`	A selector-sharing configuration with a required connector factory returning a SubjectLike<A>.
`ConnectableConfig<A>`	An explicit-connection configuration with a required connector when an object is supplied, and optional resetOnDisconnect. This name comes from the implementation; it is not a root-exported RxJS 7.8.2 type.
`Connectable<A>`	An Observable<A> that also exposes connect(): Subscription. The modern connectable() factory returns this shape.
`ConnectableObservable<A>`	The legacy connectable class shape, carrying older connection and refCount machinery. It is distinct from the minimal modern Connectable<A> interface.
`Subject<A> / SubjectLike<A>`	A Subject can receive next/error/complete and be subscribed to as an Observable. SubjectLike names the structural observer-and-subscribable contract. Its subtype determines replay and terminal behavior.
`Observer<A> / Subscription / Unsubscribable`	An Observer handles next/error/complete. A Subscription manages teardown. Unsubscribable is the smaller contract with unsubscribe(): void.
`GroupedObservable<K, A>`	An Observable<A> with a key: K property. The shorthand (A* & { readonly key: K }) preserves both stream shape and the group key.
`TimeoutConfig<A, B*> / TimeoutInfo<A>`	TimeoutConfig names first?, each?, scheduler?, with?, and meta?. TimeoutInfo carries meta, seen, and lastValue. The shorthand specializes its fallback ObservableInput parameter to B*.
`Notice<A>`	The structural N/E/C notification union defined in the legend. materialize returns Notification instances compatible with that union; dematerialize accepts the structural union.

Notation	Read it as
`AjaxConfig / AjaxResponse<A>`	AjaxConfig chooses the XMLHttpRequest request and progress options. AjaxResponse<A> is the wrapper whose response field contains A; selecting a type does not validate a server response.
`WebSocketSubjectConfig<A> / WebSocketSubject<A>`	The config supplies connection, serialization, and deserialization options. The result is both a stream of received A values and an outgoing next(value) interface.
`EventTarget / AddEventListenerOptions`	The selected DOM event target and its capture, once, and passive listener options. fromEvent also supports other event-source protocols.
`Request / RequestInit / Response`	The platform fetch request, request-options, and response types. The response body is consumed separately unless the fromFetch selector includes it.
`Date`	A JavaScript absolute date value, interpreted by the relevant operator against its supplied clock.
`Record<string, A> / A[K] / keyof A`	A string-keyed record; the value type at property K; and the permitted property-key types of A. Known-property shorthand assumes K extends keyof A.
`Exclude<A, B> / readonly`	Exclude removes union members assignable to B; readonly forbids assignment through that property in TypeScript. Neither performs runtime validation.

Alphabetical operator index

Names with an -operator suffix distinguish legacy pipeable forms from standalone functions. Select an entry to open its teaching card.

ajax 4
animationFrames 10
audit 32
auditTime 32
bindCallback 4
bindNodeCallback 5
buffer 17
bufferCount 18
bufferTime 18
bufferToggle 19
bufferWhen 19
catchError 56
combineAll 45
combineLatest 13
combineLatest-operator 45
combineLatestAll 41
combineLatestWith 46
concat 14
concat-operator 46
concatAll 42
concatMap 20
concatMapTo 20
concatWith 47
connect 54
connectable 55
count 66
debounce 33
debounceTime 33
defaultIfEmpty 64
defer 5
delay 59
delayWhen 59
dematerialize 59
distinct 33
distinctUntilChanged 34
distinctUntilKeyChanged 34
elementAt 34
empty 5
EMPTY 10
endWith 47
every 64
exhaust 21
exhaustAll 42
exhaustMap 21
expand 22
filter 35
finalize 62
find 64
findIndex 65
first 35
flatMap 31
forkJoin 14
from 6
fromEvent 6
fromEventPattern 7
fromFetch 10
generate 7
groupBy 22
ignoreElements 35
iif 9
interval 8
isEmpty 65
last 36
map 23
mapTo 23
materialize 60
max 66
merge 15
merge-operator 48
mergeAll 43
mergeMap 24
mergeMapTo 24
mergeScan 25
mergeWith 48
min 67
multicast 51
NEVER 11
never 11
observeOn 60
of 8
onErrorResumeNext 17
onErrorResumeNext-operator 58
onErrorResumeNextWith 58
pairs 11
pairwise 25
partition 15
partition-operator 26
pluck 26
publish 52
publishBehavior 52
publishLast 53
publishReplay 53
race 16
race-operator 49
raceWith 49
range 8
reduce 67
refCount 55
repeat 63
repeatWhen 63
retry 57
retryWhen 57
sample 36
sampleTime 37
scan 27
scheduled 12
sequenceEqual 65
share 54
shareReplay 56
single 37
skip 37
skipLast 38
skipUntil 38
skipWhile 38
startWith 44
subscribeOn 60
switchAll 43
switchMap 28
switchMapTo 28
switchScan 27
take 39
takeLast 39
takeUntil 39
takeWhile 40
tap 58
throttle 40
throttleTime 41
throwError 9
throwIfEmpty 66
timeInterval 61
timeout 61
timeoutWith 62
timer 9
timestamp 61
toArray 62
using 12
webSocket 13
window 29
windowCount 29
windowTime 30
windowToggle 30
windowWhen 31
withLatestFrom 44
zip 16
zip-operator 50
zipAll 50
zipWith 51